

דו"ח מכין למעבדה 5

Scalar Pipelined RISC-V CPU Architecture

מעבד RV32IM - מימוש Single-Cycle ומימוש Pipeline בן 5 שלבים

מעבדת ארכיטקטורת מעבדים מתקדמת ומאיצי חומרה

361.1.4693

אדר שפירא 209580208

יהונתן דדכה 211468582

23.08.2026

שם הקורס

מספר קורס

מגשים

תאריך הגשה

1. מטרות המעבדה ורקע תיאורטי

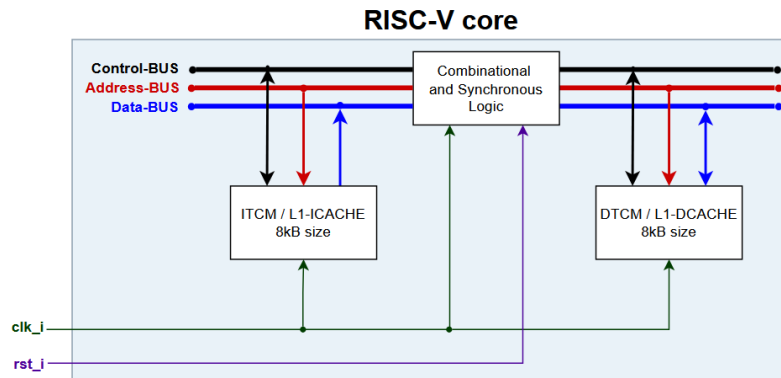
מטרת מעבדה זו היא לתכנן, לסנתז ולנתח מעבד תואם RISC-V, ולהעלות אותו ממימוש single-cycle למימוש scalar pipelined בן חמישה שלבים. העבודה התמקדה בארבעה נושאים:

- תכנון, סינתזה וניתוח של מעבד תואם RISC-V.
- מימוש מיקרו־ארכיטקטורת pipeline הכוללת יחידת forwarding מלאה ויחידת גילוי hazards בגישת combinational check.
- ההבדל בין CPU ל-MCU: הליבה שתכננו היא מעבד בלי היקפיים, בלי פסיקות ובלי בקרי קלט/פלט. יש בה מסלול נתונים, יחידת בקרה ושני זיכרונות צמודים בלבד.
- מבנה הזיכרון ב-FPGA: את ה-ITCM ואת ה-DTCM מימשנו באמצעות בלוקי M9K המשובצים ברכיב ארכיטקטורת RISC-V היא ארכיטקטורת Harvard: זיכרון ההוראות וזיכרון הנתונים מופרדים לחלוטין, ולכל אחד מהם אפיק משלו. ההפרדה מעלה את קצב התפוקה (throughput), מפני שאפשר לשלוף הוראה ולגשת לנתון באותו מחזור שעון. היא גם מפשטת את הלוגיקה של שלב ה-MEM ב-pipeline, שכן אין תחרות (structural hazard) בין שליפת הוראה לבין גישה לזיכרון נתונים.

2. הגדרת המשימה וארכיטקטורת המערכת

2.1 ארכיטקטורת המערכת הכוללת

המערכת בנויה סביב ליבת RISC-V, המחוברת לשני זיכרונות צמודים: ITCM לאחסון הקוד, ו-DTCM לאחסון הנתונים. הליבה מקבלת שעון יחיד (clk_i) ואיפוס (rst_i), כאשר האיפוס מחווט ללחצן KEY0 ומחזיר את ה-PC להוראה הראשונה בתוכנית.



איור 1: ארכיטקטורת המערכת: ליבת RISC-V עם זיכרונות ITCM ו-DTCM צמודים

הן הישות העליונה והן ליבת ה-RISC-V מומשו כתכנ מבני (structural), כלומר כחיווט מפורש של תת־מודולים ולא כתיאור התנהגותי מונוליתי. כך נשמרת התאמה אחד־לאחד בין קוד ה-VHDL לבין תרשים המיקרו־ארכיטקטורה, ובעזרת ה-RTL Viewer אפשר לאתר בקלות כל בלוק שמופיע בתרשים.

2.2 סביבת הפיתוח והכלים

רכיב	ערך	הערה
כרטיס פיתוח	DE2-115	-
רכיב FPGA	EP4CE115F29C7	Cyclone IV E
כלי סינתזה	Quartus Prime 21.1.0 Lite	קלט VHDL-2008
כלי סימולציה	ModelSim	מחזור שעות 100 ns ב-testbench
מודל זהב	RARS	השוואת DTCM.mem מול DTCM.h
מקור שעות על הכרטיס	50 MHz	נכנס ל-clk_i
שעות הליבה MCLK	25 MHz	נגזר ב-PLL: $1/2 \times 50$

טבלה 1: סביבת הפיתוח והכלים ששימשו בעבודה

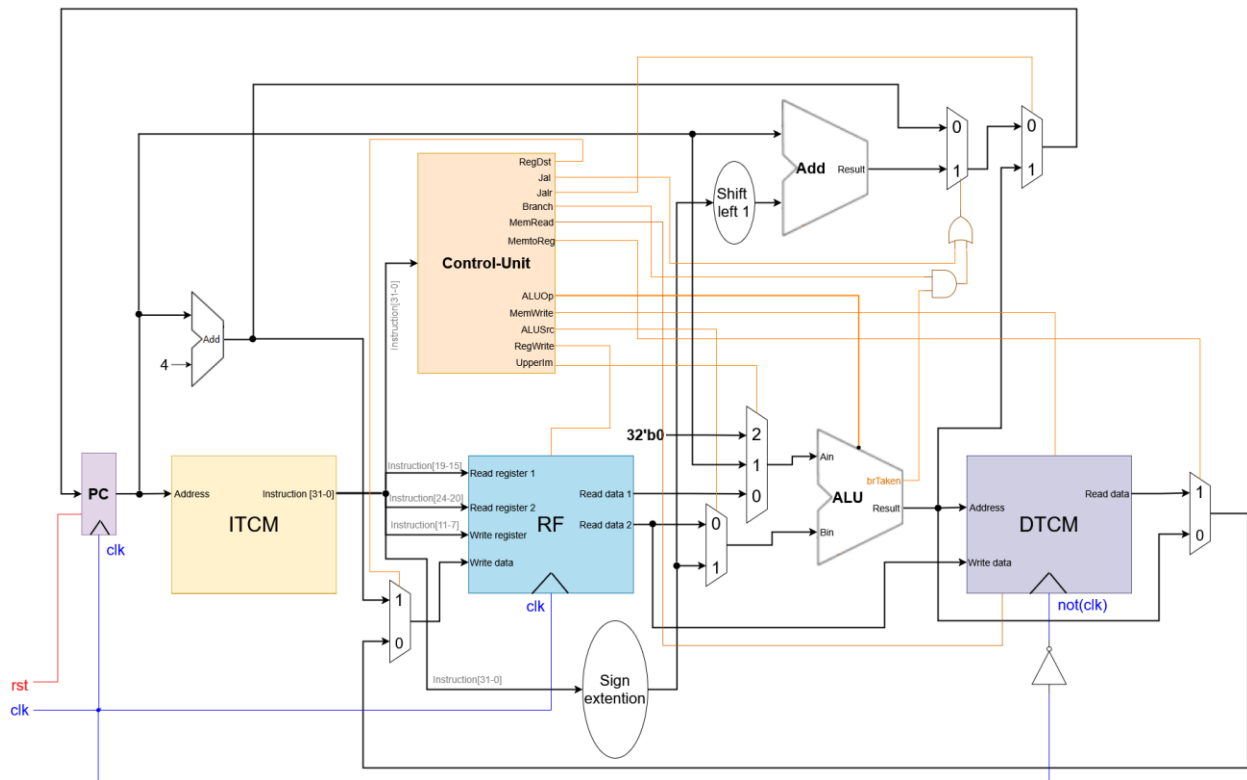
התכן משתמש בקומפילציה מותנית (cond_compilation_package.vhd) כדי לעבוד בשתי סביבות מאותו קוד מקור. כאשר $G_MODELSIM = 1$ הזיכרונות נטענים מקבצי hex וה-PLL אינו מיוצר כלל, והשעות מוזן ישירות מה-testbench. כאשר $G_MODELSIM = 0$ נוצר רכיב ALTPLL הגוזר את שעות הליבה מהמתנד שעל הכרטיס. שאר הפרמטרים בחבילה קובעים את גודל ה-TCM (8 KiB), את רוחב ה-PC ואת רזולוציית הכתובות ($G_WORD_GRANULARITY = True$, כלומר כתובת ייחודית לכל מילה).

3. חלק 1 - מיקרו־ארכיטקטורת Single-Cycle RV32IM

3.1 נקודת המוצא: Single-Cycle RV32I

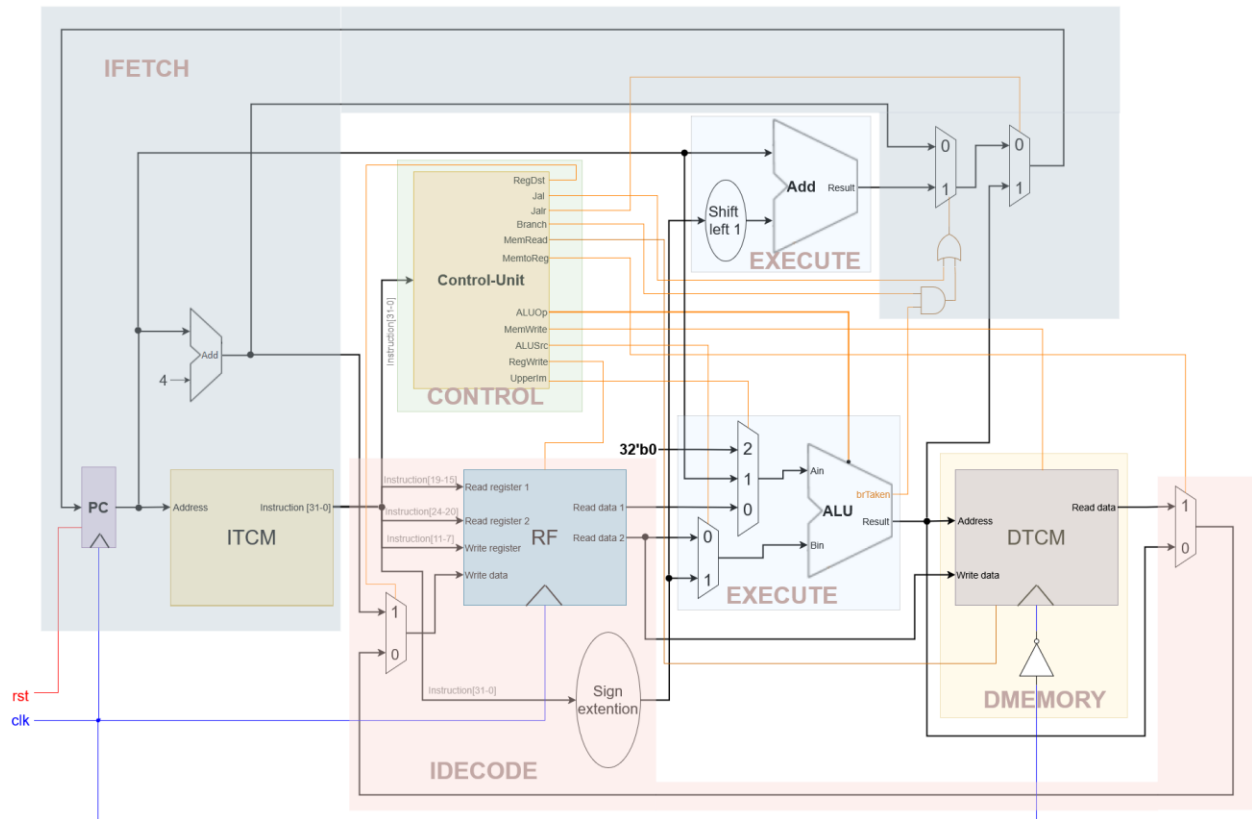
כנקודת מוצא קיבלנו מיקרו־ארכיטקטורת single-cycle התומכת ב-RV32I, ה-ISA הבסיסי הלא־מיוחס. במימוש זה כל הוראה מסיימת את ביצועה במחזור שעון יחיד. ה-PC מספק את כתובת ההוראה, ה-ITCM מחזיר את ההוראה, יחידת הבקרה מפענחת אותה, קובץ האוגרים נקרא, ה-ALU מבצע את החישוב, ה-DTCM נגיש לקריאה או לכתיבה, והתוצאה נכתבת בחזרה לקובץ האוגרים, הכול בתוך אותו מחזור.

RISC-V RV32I Single-Cycle uArchitecture (supports the base Integer unprivileged ISA) ©Hanan Ribo



איור 2: מיקרו־ארכיטקטורת Single-Cycle RV32I: נקודת המוצא של המשימה

המימוש מחולק לחמישה תת־מודולים פונקציונליים המהווים גם את הבסיס לחלוקה לשלבי ה-pipeline בחלק 2: IFETCH, IDCODE, CONTROL, EXECUTE ו-DMEMORY.



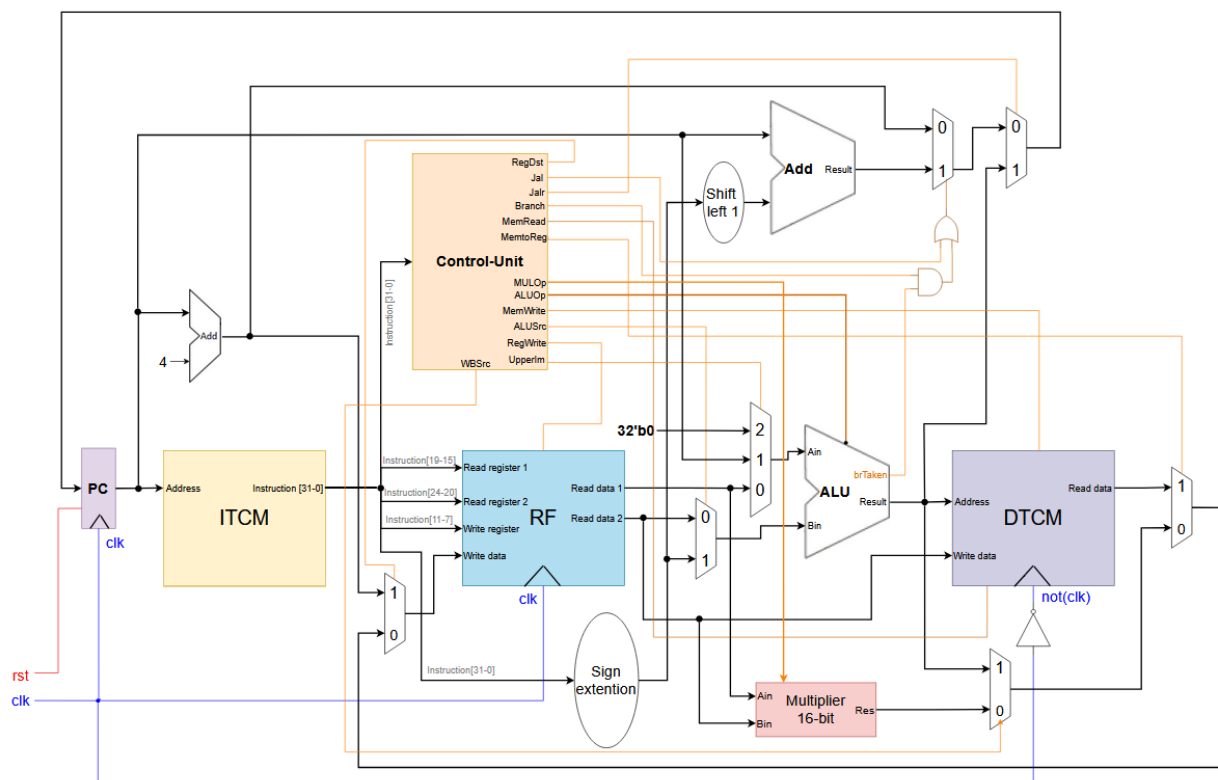
איור 3: חלוקת המיקרו־ארכיטקטורה לתת־מודולים (submodules)

ה-DTCM נכתב על השעון ההפוך ($\text{not}(\text{clk})$). הסיבה: במימוש single-cycle, כתובת הזיכרון היא תוצאת ה-ALU, שמתייצבת רק בסוף המחזור. כתיבה על העלייה הבאה של השעון הייתה מאחרת את הנתון במחזור שלם. הכתיבה על הירידה מקצה חצי מחזור לייצוב הכתובת וחצי מחזור לפעולת הזיכרון עצמה. להחלטה זו יש השלכה ישירה על ניתוח התזמון: המסלול הקריטי של המימוש הוא מסלול חצי מחזור, ולכן $f_{\max}$ נקבע לפי $1 / (2 \cdot t_{\text{path}})$ ולא לפי $1 / t_{\text{path}}$.

3.2 הרחבת M: מימוש פקודת mul

משימת חלק 1 היא להעלות את המיקרו־ארכיטקטורה מ-RV32I ל-RV32IM. הרחבת M מגדירה פעולות כפל וחילוק (MULDIV), אך דרישת המשימה מסתפקת בתמיכה חלקית: פקודת mul בלבד מתוך קבוצת הכפל (mul, mulh, mulhu, mulhsu).

סמנטיקת הפקודה כפי שהוגדרה במשימה: mul rd, rs1, rs2 מכפילה את חצאי המילה התחתונים (16 ביט) של האוגרים rs1 ו- rs2 , וכותבת תוצאה בת 32 ביט לאוגר rd . הפקודה היא מסוג R-type, ולכן כל אותות הבקרה האחרים (MemtoReg , RegWrite וכו') זהים לאלו של פעולת ALU רגילה. ההרחבה מתבטאת אך ורק בקידוד ALUOp חדש ובנתיב נתונים נוסף.



איור 4: מיקרו־ארכיטקטורת Single-Cycle RV32IM (MULDIV partial): הכופל בן 16 הביט משולב לצד ה־ALU, ומרבב חדש בוחר בין שתי התוצאות

3.2.1 אלגוריתם הכפל

האלגוריתם הנדרש מפרק כפל 16×16 לארבעה מכפלים חלקיים בני 8×8 , כך שהסינתזה תמפה אותם ישירות על הכופלים המשובצים (embedded 9-bit multipliers) של ה-FPGA במקום לבנות כופל מלוגיקה. הפירוק מתבצע בשני שלבים:

- $P0 = A_{low} \times B_{low}$
- $P1 = A_{low} \times B_{high}$
- $P2 = A_{high} \times B_{low}$
- $P3 = A_{high} \times B_{high}$

Stage 1

- $M = P1 + P2$
- $RESULT = P0 + (M \ll 8) + (P3 \ll 16)$

Stage 2

המימוש שלנו מתרגם את המשוואות אחד-לאחד: כל אופרנד מפוצל לשני חצאים בני 8 ביט, ארבעת המכפלים החלקיים מחושבים במקביל, הסכום האמצעי $M = P_1 + P_2$ מחושב ברוחב 17 ביט (ביט נוסף לנשא), ולבסוף שלושת האיברים מיושרים לרוחב 32 ביט ומחוברים. כדי לוודא שהמיפוי אכן התבצע כמתוכנן בדקנו את דוח ה-Area, והוא מדווח על 4 יחידות embedded 9-bit multiplier בשני המימושים, בדיוק כמספר המכפלים החלקיים. כלומר הסינתזה מיפתה את הכפל לחומרה הייעודית ולא ללוגיקה.

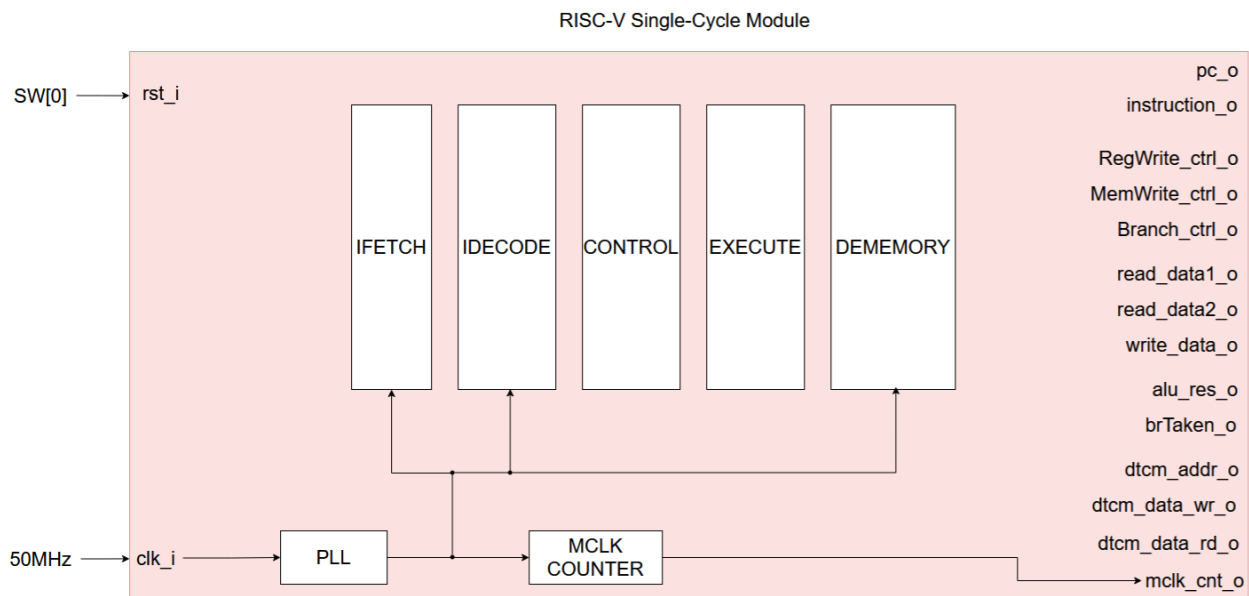
3.2.2 השינויים שנדרשו בקבצי התכנ

קובץ	השינוי שבוצע
const_package.vhd	הוספת תבנית הזיהוי INST_MUL ומסכת הביטים INST_MUL_MASK, והוספת הקידוד ALU_MUL לרשימת פעולות ה-ALU המשותפת ל-CONTROL ול-EXECUTE.
CONTROL.vhd	זיהוי הפקודה לפי INST_MUL_MASK/INST_MUL והנעת ALUOp = ALU_MUL. שאר אותות הבקרה נותרו ללא שינוי, שכן mul היא R-type.
MUL16.vhd	מודול חדש - הכופל $16 \times 16 \rightarrow 32$ על בסיס ארבעה מכפלים חלקיים.
EXECUTE.vhd	יצירת מופע של MUL16 ובחירת פלטו כ-alu_res כאשר ALUOp = ALU_MUL.
aux_package.vhd	הוספת הצהרת הרכיב MUL16 ושינוי שם הליבה מ-RV32I_CORE ל-RV32IM_CORE.

טבלה 2: סיכום השינויים במעבר מ-RV32I ל-RV32IM במימוש ה-single-cycle

3.3 הישות העליונה ותרשים בלוקים

הישות העליונה RV32IM_CORE מחווטת את חמשת תת-המודולים ומוציאה החוצה קבוצת אותות ניפוי המשמשים הן את חלון הגלים ב-ModelSim והן את הוולידציה ב-Signal-Tap על הכרטיס: ה-PC, ההוראה, אותות הבקרה המרכזיים (RegWrite_ctrl_o, MemWrite_ctrl_o, Branch_ctrl_o), נתוני קובץ האוגרים, תוצאת ה-ALU, אפיקי ה-DTCM ומונה מחזורי השעון mclk_cnt_o.



איור 5: הישות העליונה של מימוש ה-Single-Cycle: RV32IM_CORE ואותות הניפוי שלה

במימוש single-cycle כל הוראה נמשכת מחזור אחד בדיוק, ולכן $IPC = 1$ בהגדרה ומונה מחזורי השעון בסוף הריצה שווה למספר ההוראות שבוצעו בפועל.

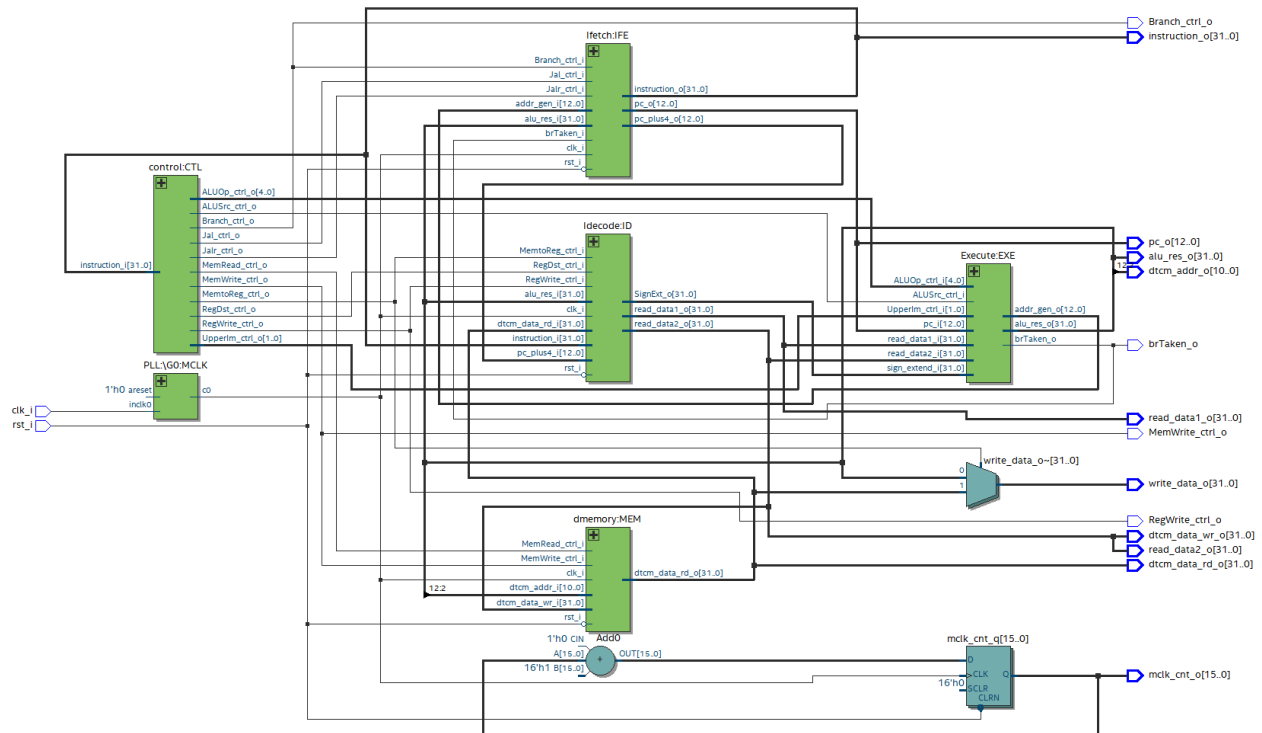
3.4 תיאור קבצי ה-HDL

קובץ	תיאור
cond_compilation_package.vhd	פרמטרי קומפילציה מותנית: מיתוג FPGA/ModelSim, גודל ה-TCM (1–8 KiB), רוחבי PC ו-MA, ויחסי החלוקה והכפל של ה-PLL.
const_package.vhd	תבניות ה-opcode ומסכות ההוראות שבהן משתמש CONTROL, וקידוד ALUOp המשותף ל-CONTROL ול-EXECUTE (כולל ALU_MUL שנוסף עבור הרחבת M).
MUL16.vhd	כופל $16 \times 16 \rightarrow 32$ של חצאי המילה התחתונים של rs2/rs1, הבנוי מארבעה מכפלים חלקיים בני 8 ביט לפי $RESULT = P0 + ((P1+P2) \ll 8) + (P3 \ll 16)$, כך ש-Quartus ממפה אותו לכופלים המשובצים.
CONTROL.vhd	מפענח צירופי (combinational decoder); מזהה mul באמצעות $INST_MUL_MASK/INST_MUL$ ומניע $ALUOp = ALU_MUL$.
EXECUTE.vhd	ה-ALU (חיבור/חיסור, פעולות לוגיות, barrel shifters, השוואות) ולוגיקת ההסתעפות; יוצר מופע של MUL16 ובוחר את תוצאתו כ-alu_res עבור ALU_MUL.
IFETCH.vhd	אוגר ה-PC, מחבר PC+4, מרבב ה-PC הבא (jalr/jal/branch) וה-ITCM (altsyncram).
IDECODE.vhd	קובץ אוגרים 32×32 , חילוף שדות ההוראה, כל חמש תבניות ה-immediate והרחבת הסימן, ומרבב ה-write-back.
DMEMORY.vhd	ה-DTCM (altsyncram), הנכתב על השעון ההפוך.
aux_package.vhd	הצהרות הרכיבים של כל התכן.
PLL.vhd	הגזור את שעון הליבה בן 25 MHz מהמתנד בן 50 MHz שעל הכרטיס; נוצר רק כאשר $G_MODELSIM = 0$.
RV32IM_CORE.vhd	הישות העליונה המבנית; מחווטת את חמשת תת-המודולים וחושפת את אותות הניפוי ל-Signal-Tap.

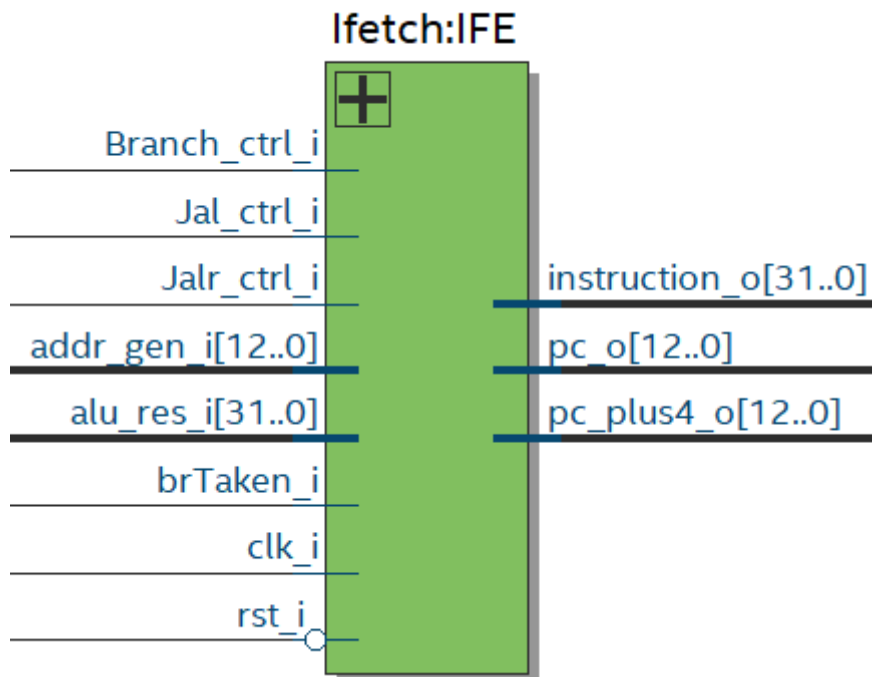
טבלה 3: תיאור קבצי ה-HDL של מימוש ה-Single-Cycle RV32IM

3.5 תוצאות RTL Viewer

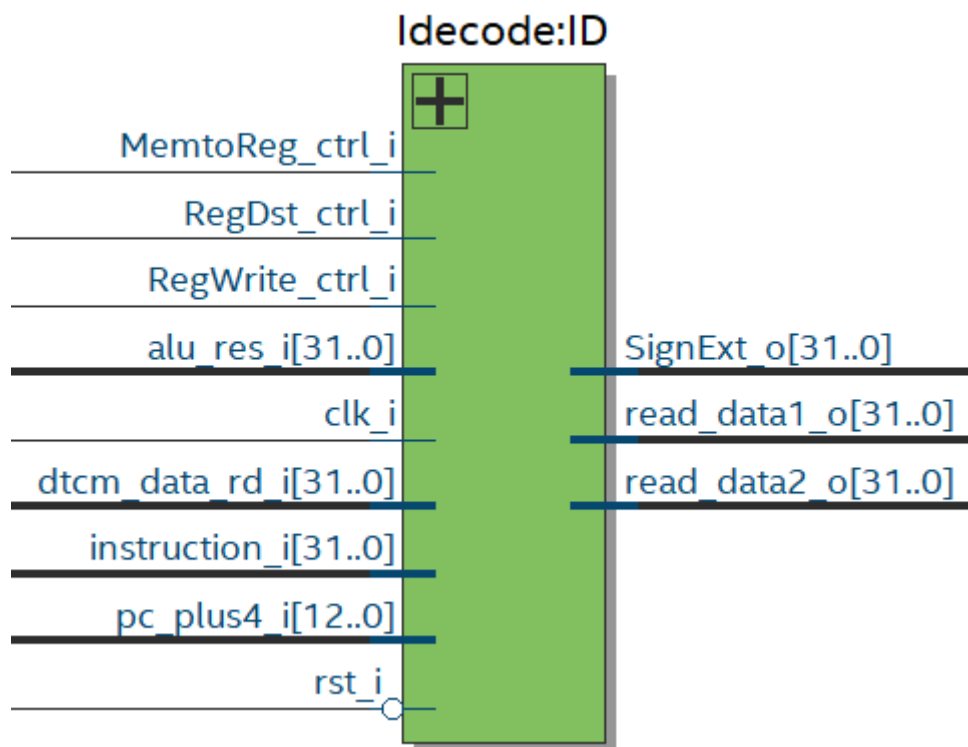
לאחר הסינתזה ב-Quartus בחנו את התכן ב-RTL Viewer כדי לוודא שהמבנה שהתקבל תואם לתרשים המיקרו-ארכיטקטורה. מכיוון שהתכן העליון מבני, הבלוקים בתצוגה מתאימים אחד-לאחד לבלוקים שבאיור 4.



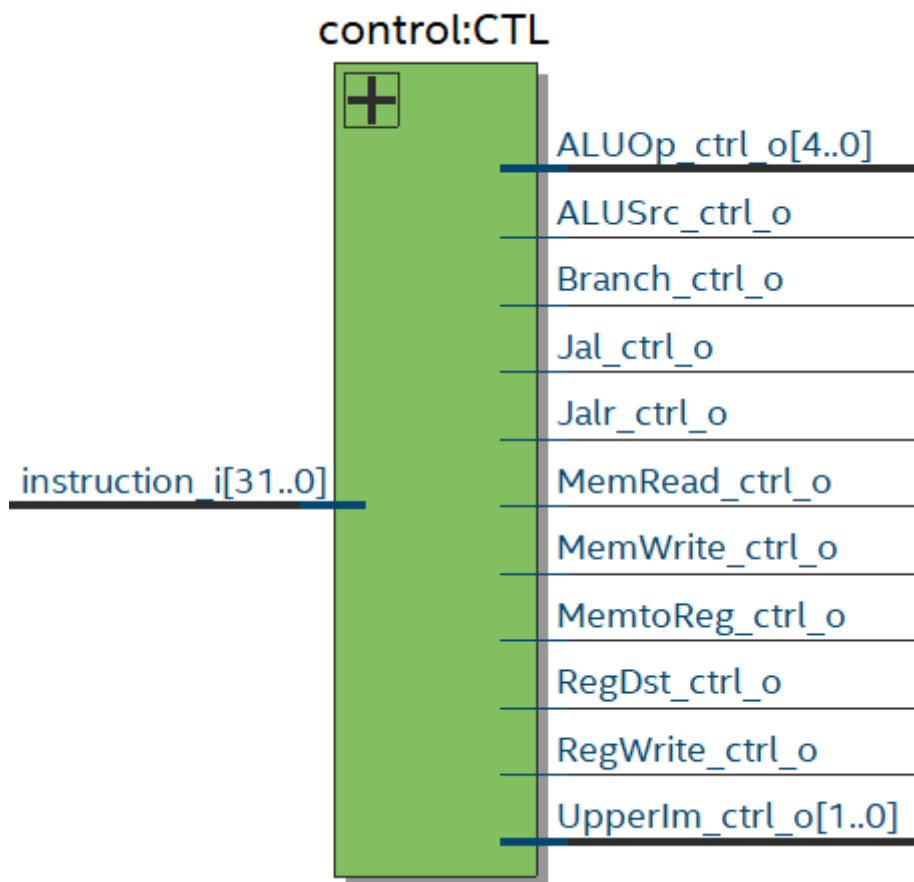
איור 6: הרמה העליונה של RV32IM_CORE ב-RTL Viewer: חמשת תתי-מודולים ואפיקי החיבור ביניהם



איור 7: מודול IFETCH ב-RTL Viewer

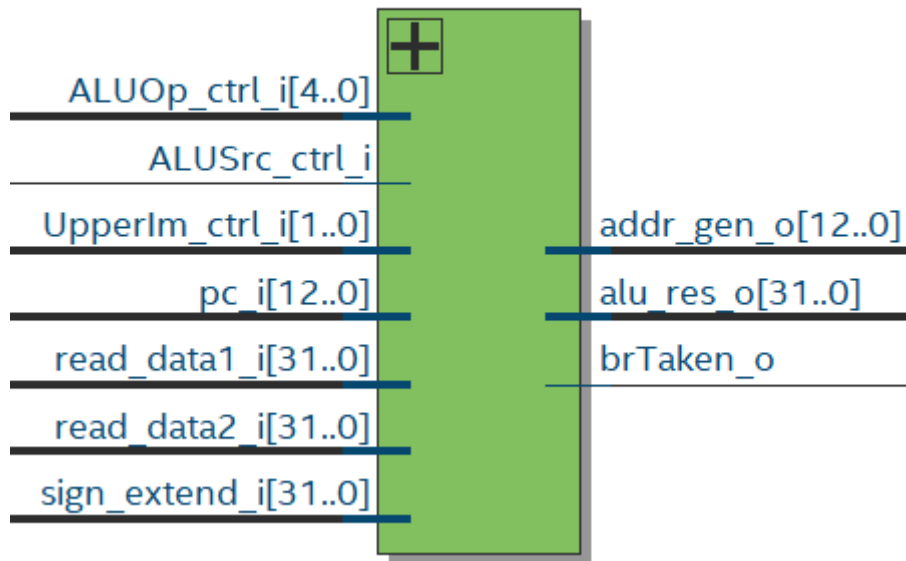


איור 8: מודול IDECODE ב-RTL Viewer

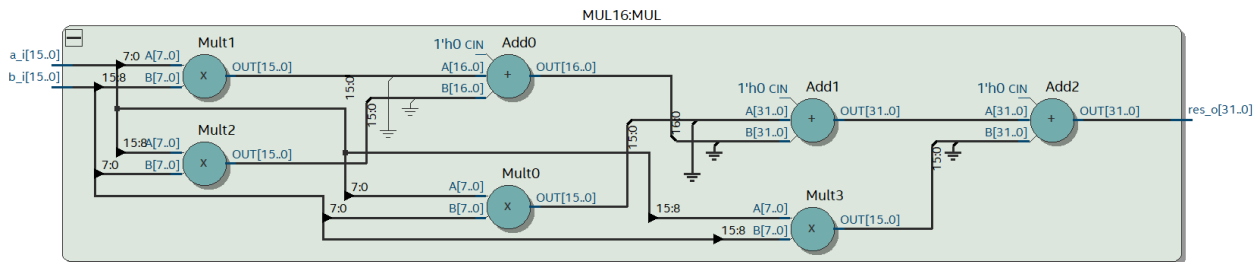


איור 9: מודול CONTROL ב-RTL Viewer

Execute:EXE

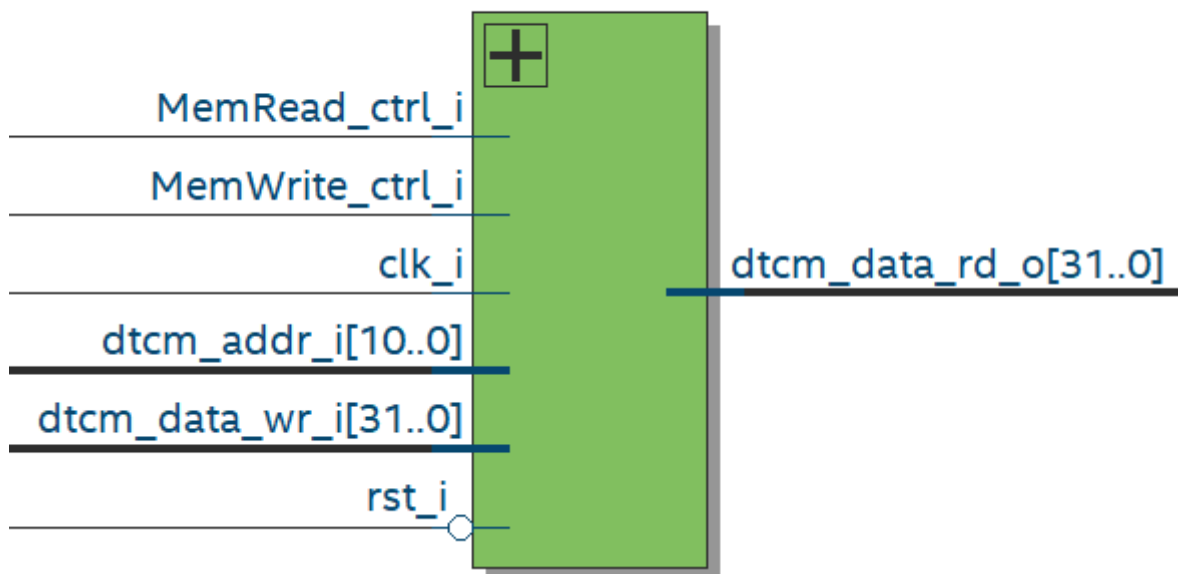


איור 10: מודול EXECUTE ב-RTL Viewer



איור 11: מודול MUL16 ב-RTL Viewer: ארבעת המכפלים החלקיים בני 8 ביט, סכום האמצע והחיבור הסופי

dmemory:MEM



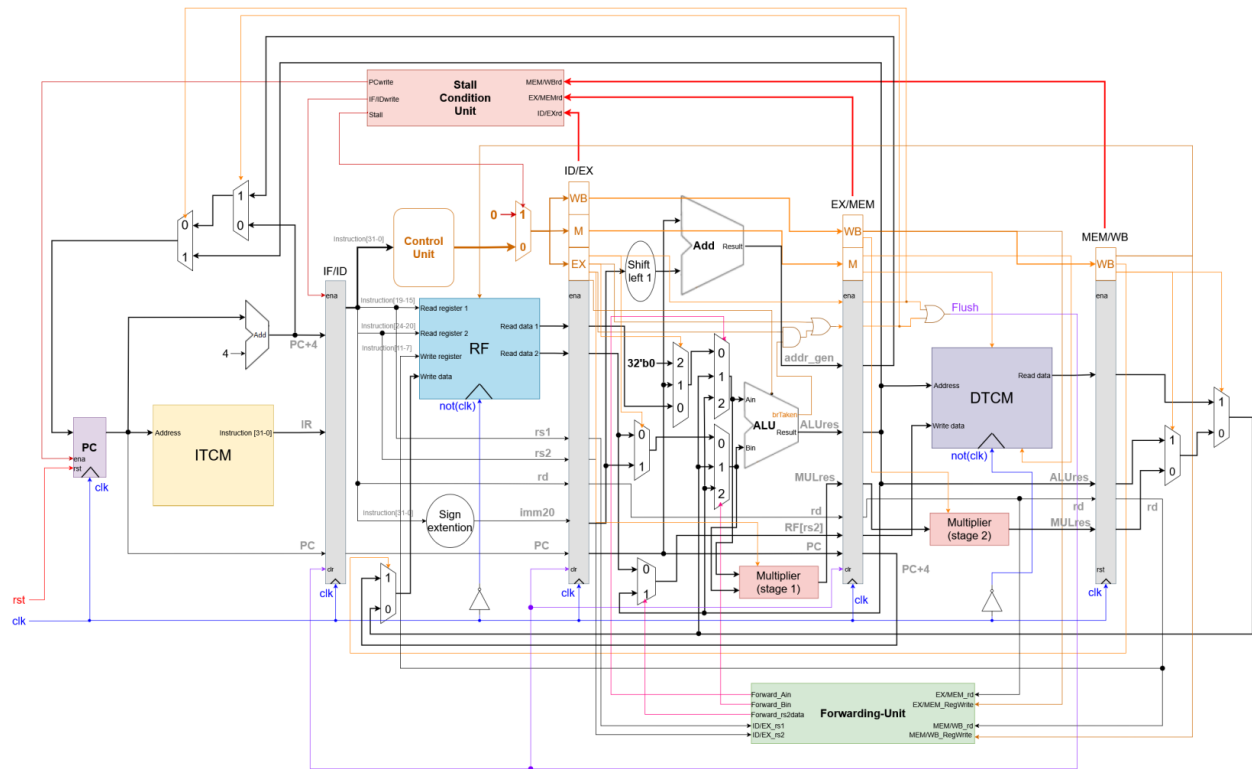
איור 12: מודול DMEMORY ב-RTL Viewer

4. חלק 2 - מיקרו־ארכיטקטורת Pipelined RV32IM

בחלק השני של המשימה הרחבנו את מיקרו־ארכיטקטורת ה-RV32IM single-cycle שתוכננה בחלק 1 למימוש pipeline סקלרי בן חמישה שלבים. במקום שהוראה אחת תעבור את כל המסלול הארוך במחזור אחד, המסלול נחתך לחמישה קטעים קצרים המופרדים באוגרי pipeline, וחמש הוראות שונות מתקדמות בו־זמנית, כל אחת בשלב אחר. תדר השעון נקבע כעת לפי השלב האיטי ביותר בלבד ולא לפי סכום כל השלבים.

בהשוואה לתיאור הראשוני של חלק 1 (סעיף 3.2.1), הכופל בן 16 הביט אינו נשאר עוד יחידה אחת בשלב EX: כדי לא להאריך את שלב ה-EX בעומק הלוגי של סכימת שלושת האיברים, הכפל פוצל לשני שלבי pipeline נפרדים.

RISC-V RV32I Single-Thread pipeline uArchitecture (supports the base Integer unprivileged ISA) ©Hananya Ribo



איור 13: מיקרו־ארכיטקטורת Pipelined RV32IM (MULDIV partial) בת חמישה שלבים

4.1 חלוקה לחמישה שלבים ואוגרי ה-pipeline

את חלוקת התכן ביססנו על תת-המודולים הקיימים מחלק 1: כל תת-מודול הפך לשלב pipeline, ואוגר ה-pipeline שבסופו מומש בתוך אותו מודול ולא כישות נפרדת. כך נשמר התכן המבני של הרמה העליונה, המחוותת חמישה שלבים ושתי יחידות בקרת pipeline בדיוק כמו בתרשים. לעומת חלק 1, שלב ה-EX וה-MEM התפצלו כל אחד למחצית מודול הכפל (MULT_1 ו-MULT_2 בהתאמה), ונוסף שלב WB עצמאי שמרכז את מרבב ה-write-back שבחלק 1 היה חלק מ-IDECODE.

#	שלב	מודול VHDL	מה מתבצע בשלב	אוגר ה-pipeline שבסופו
1	IF	IFETCH.vhd	אוגר ה-PC, מחבר PC+4, מרבב ה-redirect, קריאה מה-ITCM	IF/ID
2	ID	IDECODE.vhd CONTROL.vhd	פענוח, קריאת קובץ האוגרים, ייצור immediate והרחבת סימן	ID/EX
3	EX	EXECUTE.vhd MULT_1.vhd	מרבבי forwarding, ה-ALU, MUL16, מחבר כתובת ההסתעפות, ארבעת המכפלים החלקיים P0-P3	EX/MEM
4	MEM	DMEMORY.vhd MULT_2.vhd	גישה ל-DTCM, צירוף תוצאות הכפל, הכרעת הסתעפויות וקפיצות	MEM/WB
5	WB	WRITEBACK.vhd	מרבב תלת כיווני (jal/jalr/PC+4): תוצאת ALU או כפל נתון מה-DTCM, וכתיבה לקובץ האוגרים	-

טבלה 4: מיפוי חמשת שלבי ה-pipeline למודולי ה-VHDL ולאוגרי ה-pipeline

קובץ האוגרים נקרא בשלב ID ונכתב בשלב WB, ולכן שני הצדדים ממוקמים באותו מודול (IDECODE). פתרון זה גם מאפשר לממש בתוך המודול את עקיפת הקריאה (read bypass): כאשר הוראה בשלב WB כותבת לאותו אוגר שהוראה בשלב ID קוראת ממנו, הקריאה מקבלת את הערך החדש (wb_write_data_i) ולא את התוכן הישן שב-RF. בלי עקיפה זו היה נדרש עוד מנגנון forwarding או עוד stall.

4.2 data hazards ויחידת ה-forwarding

data hazard מסוג RAW מתרחש כאשר הוראה זקוקה לתוצאה של הוראה קודמת שטרם נכתבה לקובץ האוגרים. במקום להשהות את ה-pipeline, יחידת ה-forwarding מזרימה את התוצאה ישירות מאוגר ה-pipeline שבו היא כבר קיימת אל כניסות ה-ALU.

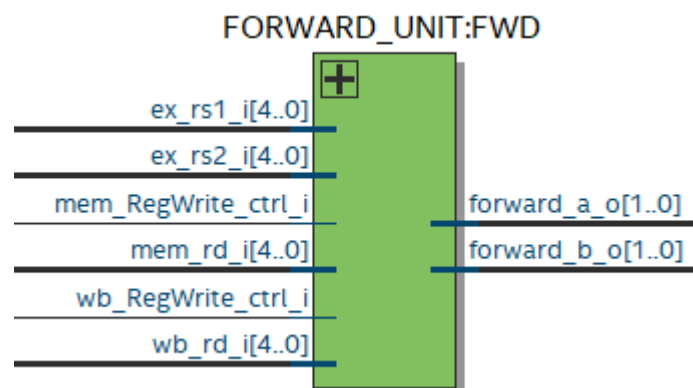
היחידה FORWARD_UNIT.vhd משווה את rs1 ו-rs2 של ההוראה שבשלב EX מול אוגרי היעד של ההוראות שבשלב MEM ו-WB, ומניעה שני אותות בחירה בני שני ביט (forward_a_o, forward_b_o) המזינים את מרבבי הכניסה של ה-ALU בתוך EXECUTE.vhd:

קידוד	שם	מקור הנתון	מתי
00	FWD_NONE	אוגר ה-ID/EX (הערך שנקרא מקובץ האוגרים)	אין תלות
10	FWD_MEM	mem_forward_data_o של mem_forward_data_o:DMEMORY תוצאת ה-ALU או תוצאת שלב 2 של הכפל (MULT_2), לפי Mul_ctrl_i	תלות במרחק 1
01	FWD_WB	write_data_o של WRITEBACK (תוצאת ALU-או-כפל, נתון DTCM, או PC+4)	תלות במרחק 2

טבלה 5: קידוד מרבבי ה-forwarding ביחידת FORWARD_UNIT

מקור ה-FWD_MEM אינו "שדה ה-ALU של אוגר EX/MEM" כפי שהיה בחלק 1, אלא הפלט הקומבינטורי mem_forward_data_o של DMEMORY. עבור הוראת mul שדה ה-alu_res של EX/MEM אינו מכיל את המכפלה כלל – המכפלה מוכנה רק בשלב MEM לאחר MULT_2 – ולכן ה-forwarding חייב להיות ממקור שכבר ביצע את הבחירה בין ALU לכפל (mem_result_w).

שני תנאים נוספים חייבים להתקיים כדי שהנתון יוזרם: ההוראה הקודמת אכן כותבת לאוגר ('1' = RegWrite), והיעד אינו x0. האוגר x0 קשיח לאפס בארכיטקטורת RISC-V, ולכן forwarding אליו היה שגוי. כמו כן נקבעה עדיפות ל-EX/MEM על פני MEM/WB: אם שתי ההוראות שלפנינו כותבות לאותו אוגר, הערך הנכון הוא של המאוחרת מביניהן, לפי הכלל "הכותב האחרון מנצח".



איור 14: RTL Viewer - יחידת FORWARD_UNIT

4.3 load-use hazard ויחידת ה-interlock

forwarding מלא מכסה כמעט את כל ה-data hazards, אך נשאר אחד שאין דרך לכסות: הוראת lw מקבלת את הנתון שלה מה-DTCM רק בסוף שלב MEM, בעוד שההוראה הצמודה אחריה זקוקה לו כבר בתחילת שלב EX שלה, כלומר באותו מחזור עצמו. אין דרך פיזית להזרים נתון אחורה בזמן, ולכן נדרש stall של מחזור אחד.

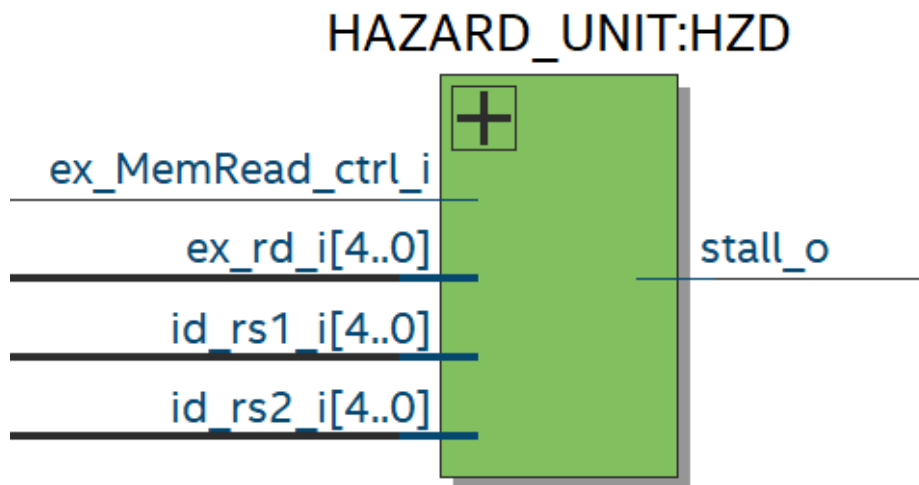
היחידה HAZARD_UNIT.vhd מממשת בדיקה צירופית טהורה (combinational checkboard), ללא מצב פנימי כלשהו:

```
stall_o = ex_MemRead_ctrl_i AND (ex_rd_i ≠ x0) AND (ex_rd_i = id_rs1_i OR ex_rd_i = id_rs2_i)
```

כאשר '1' = stall_o, הרמה העליונה מפעילה interlock של מחזור אחד:

- המודול IFETCH מקפיא את ה-PC ואת אוגר IF/ID (stall_i), כך שההוראה הצורכת מפוענחת שוב בשלב ID במחזור הבא.
- המודול DECODE מזריק bubble לאוגר ID/EX (bubble_w = stall_i or flush_i).
- הוראת ה-lw עצמה ממשיכה כרגיל במורד ה-pipeline.

מחזור stall אחד מספיק. אחריו ההוראה הצורכת עדיין בשלב ID בעוד ה-lw נמצאת ב-MEM. במחזור שאחריו הצורכת מגיעה ל-EX וה-lw נמצאת ב-WB, ואז נתון הטעינה מוזרם אליה על ידי FORWARD_UNIT בבחירה FWD_WB. שאר המרחקים מכוסים ממילא: צרכן במרחק 2 פוגש את ה-lw ב-WB בלי stall, וצרכן במרחק 3 קורא את קובץ האוגרים ב-ID בדיוק כשה-lw כותבת אליו ב-WB, מקרה שמכוסה על ידי עקיפת הקריאה שבתוך DECODE.



איור 16: RTL Viewer - יחידת HAZARD_UNIT

4.4 control hazards: הכרעת הסתעפויות בשלב MEM

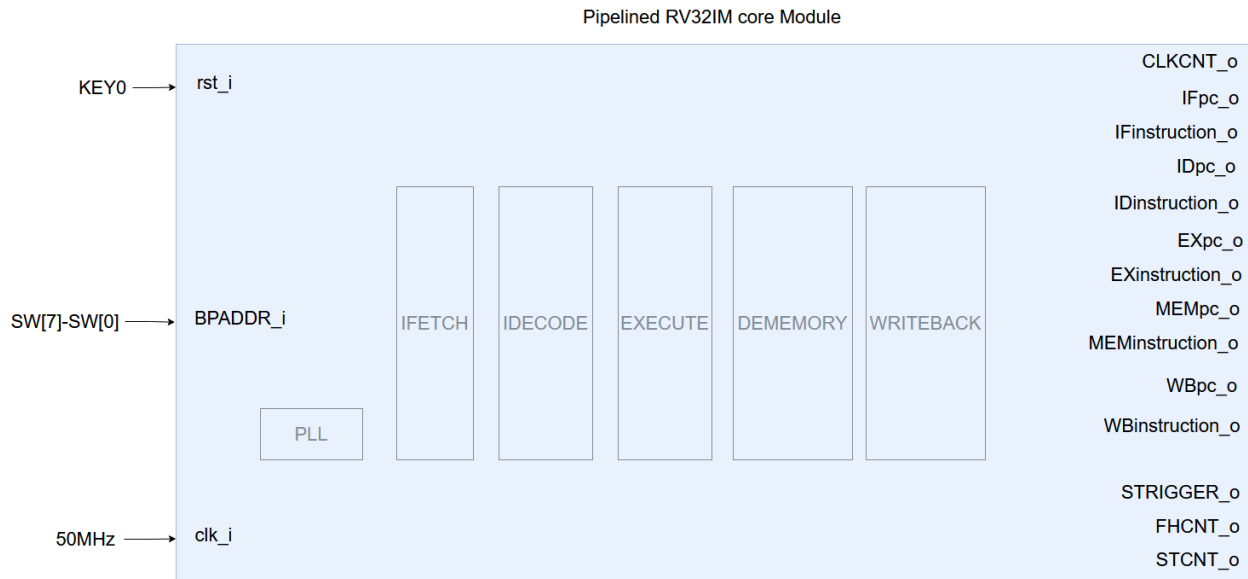
בתכן זה, הסתעפויות מותנות וקפיצות בלתי-מותנות (jalr, jal) מוכרעות בשלב הרביעי של ה-pipeline, MEM. בזמן שההוראה המסתעפת מתקדמת בין IF ל-MEM, ה-pipeline ממשיכה לשלוף באופן ספקולטיבי את ההוראות העוקבות בזיכרון. כאשר מתברר שההסתעפות נלקחת, שלוש ההוראות הצעירות ממנה, אלו שבאוגרי IF/ID, ID/EX ו-EX/MEM, שגויות ויש לבטלן. מכאן שעומק ה-flush הוא 3, וזהו המקדם depth המופיע בנוסחת ה-IPC.

כתובת ה-redirect (redirect_addr_w) נבחר מתוך אוגר ה-EX/MEM: עבור branch ו-jal זוהי תוצאת מחבר כתובת ההסתעפות, ועבור jalr זוהי תוצאת ה-ALU.

ייתכן מצב שבו stall ו-flush פעילים באותו מחזור. במקרה כזה ה-flush גובר: ההוראה שגרמה ל-redirect נמצאת ב-MEM ולכן היא מבוגרת מההוראה שגרמה ל-stall, וממילא ההוראה הצעירה עומדת להתבטל, וה-stall חסר משמעות. כלל קדימות זה מונע איבוד של אירוע redirect.

4.5 תוספות ניפוי שגיאות ו-Signal-Tap

לצורך ולידציה וחישוב IPC, הישות העליונה של ה-pipeline מרחיבה את זו של ה-single-cycle בארבע תוספות.



איור 17: הישות העליונה של ה-pipeline - RV32IM_PIPE_CORE עם מוני CLKCNT, STCNT, FHCNT ואוגר נקודת העצירה BPADDR

תפקיד	רוחב	אות
מונה מחזורי שעון מאז האיפוס - המכנה בנוסחת ה-IPC.	16	CLKCNT_o
מונה stalls; מתקדם בעליית השעון בכל מחזור שבו '1' = stall. מתאפס באיפוס.	16	STCNT_o
מונה flush; מתקדם בכל אירוע redirect. מתאפס באיפוס.	16	FHCNT_o
אוגר כתובת נקודת עצירה (ברזולוציית מילה), מוזן מ-SW7-SW0, שעל הכרטיס.	8	BPADDR_i
אות הטריגר ל-Signal-Tap, המופק מהשוואת ה-PC של שלב IF ל-BPADDR_i.	1	SPTRIGGER_o

טבלה 6: אותות הניפוי שנוספו בישות העליונה של ה-pipeline

שתי המשוואות המגדירות את המנגנון:

$$\text{STRIGGER}_o = (\text{IFpc}_o[12:2] == \text{BPADDR}_i)$$

$$\text{IPC} = \left[\text{CLKCNT}_o - (\text{STCNT}_o + 4 + \text{depth} \cdot \text{FHCNT}_o) \right] / \text{CLKCNT}_o = \text{InstructionCounter} / \text{CLKCNT}_o, \text{IPC} = 1 / \text{CPI}$$

איברי המונה מתפרשים כך: מכלל מחזורי השעון מחסירים את מחזורי ה-stall (STCNT_o), את 4 המחזורים של מילוי ה-pipeline בתחילת הריצה (pipeline fill): ההוראה הראשונה זקוקה ל-5 מחזורים כדי לפרוש, ולכן 4 מחזורים אינם מנוצלים), ואת המחזורים שאבדו בכל אירוע redirect, 3 לכל flush. מה שנותר הוא מספר ההוראות שפרשו בפועל.

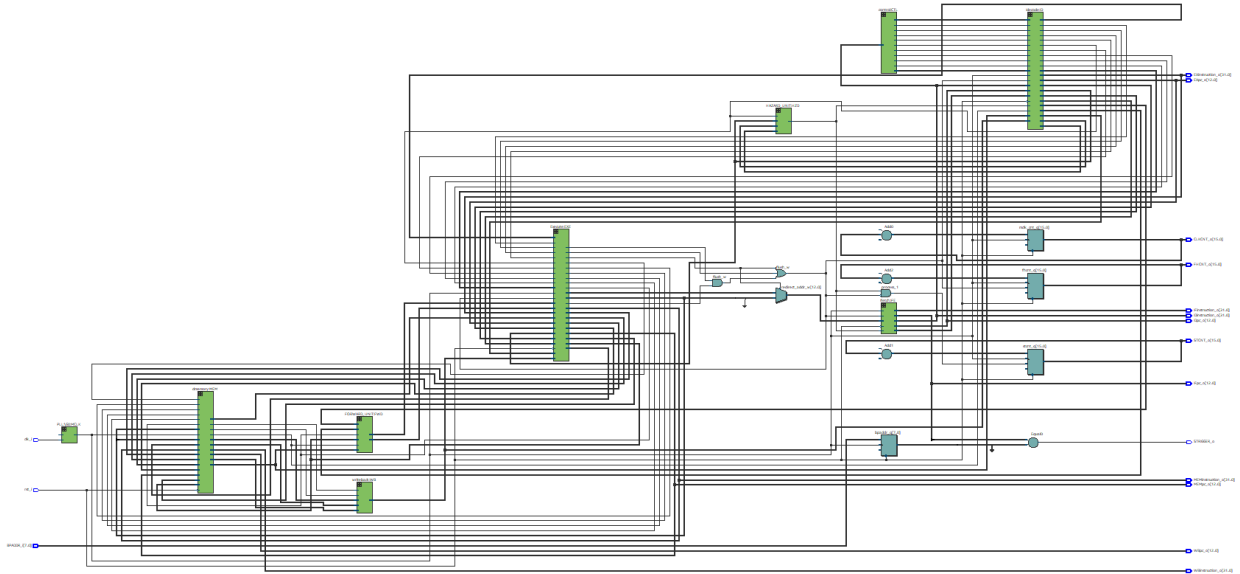
BPADDR נדגמת מ-SW7-SW0 לתוך אוגר bpaddr_q בכל עליית שעון; KEY0 משמש כאיפוס (rst_i).

4.6 תיאור קבצי ה-HDL

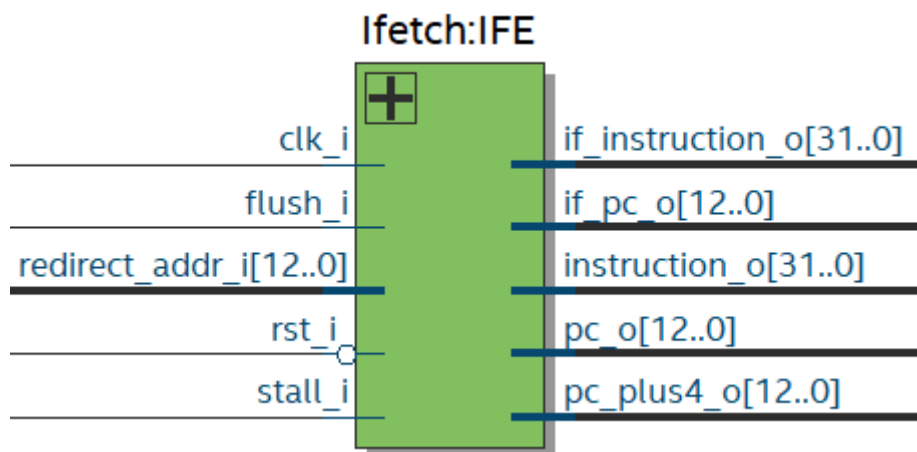
קובץ	תיאור
cond_compilation_package.vhd, const_package.vhd, PLL.vhd	זהים בתפקידם לקבצים המקבילים בחלק 1.
aux_package.vhd	הצהרות הרכיבים של כל שלבי ה-pipeline (כולל, MULT_1, MULT_2, WRITEBACK, HAZARD_UNIT, FORWARD_UNIT, RV32IM_PIPE_CORE) והישות
IFETCH.vhd	שלב IF ואוגר IF/ID. ה-PC מקבל כניסת stall_i (הקפאה), flush_i מזריק bubble ומפנה את ה-PC ליעד ה-redirect, ו-redirect_o משמש להשוואת STRIGGER.
IDCODE.vhd	שלב ID ואוגר ID/EX (נתונים, rs1/rs2/rd, immediate, ואותות בקרה). קובץ האוגרים נכתב מ-wb_write_data_i (פלט WRITEBACK) בעוד הקריאות מתבצעות ב-ID, כולל עקיפת קריאה לאותו מחזור.
CONTROL.vhd	אותו מפענח צירופי מחלק 1, היושב בשלב ID; פלטיו נעים במורד ה-pipeline יחד עם ההוראה.
EXECUTE.vhd	שלב EX ואוגר EX/MEM. מרבבי forwarding על שתי כניסות ה-ALU, ומופע MULT_1 (ארבעת המכפלים החלקיים P0–P3 מהאופרנדים המוזרמים).
MULT_1.vhd	שלב 1 של הכפל (EX, מופע MUL1): ארבעה מכפלים חלקיים 8×8 – מ-16 הביטים התחתונים של האופרנדים המוזרמים.
DMEMORY.vhd	שלב MEM ואוגר MEM/WB. מכיל את ה-DTCM ואת מופע MULT_2; מרבב mem_result_w בוחר בין תוצאת ALU לתוצאת הכפל, ומזין הן את MEM/WB והן את mem_forward_data_o.
MULT_2.vhd	שלב 2 של הכפל (MEM, מופע MUL2): מצרף את P0–P3 שעברו דרך EX/MEM ל-res_o.
WRITEBACK.vhd	מרבב ה-write-back של שלב WB (מופע WB): PC+4 (jal/jalr) / תוצאת ALU-או-כפל / נתון DTCM. הפלט (write_data_o) מזין את פורט הכתיבה של קובץ האוגרים, את עקיפת הקריאה ב-ID, ואת FWD_WB.
HAZARD_UNIT.vhd	interlock צירופי ל-load-use: משווה את rs1/rs2 של שלב ID מול אוגר היעד שבשלב EX, ומשהה מחזור אחד.
FORWARD_UNIT.vhd	משווה את rs1/rs2 של שלב EX מול אוגרי היעד שב-EX/MEM וב-MEM/WB (בתנאי 'RegWrite=1' ו-rd≠x0) ומניע את אותות בחירת מרבבי ה-forwarding.
RV32IM_PIPE_CORE.vhd	הישות העליונה המבנית; הסתעפויות וקפיצות מוכרעות ב-MEM (flush בעומק 3). כוללת את מוני CLKCNT/STCNT/FHCNT, אוגר BPADDR והשוואת STRIGGER.

טבלה 7: תיאור קבצי ה-HDL של מימוש ה-pipeline

4.7 תוצאות RTL Viewer



איור 19: RTL Viewer – הרמה העליונה של RV32IM_PIPE_CORE. תשעה מופעים ב-, control:CTL, Netlist Navigator: dmemory:MEM, Execute:EXE, FORWARD_UNIT:FWD, HAZARD_UNIT:HZD, Idecode:ID, Ifetch:IFE, writeback:WB, PLL:\G0:MCLK

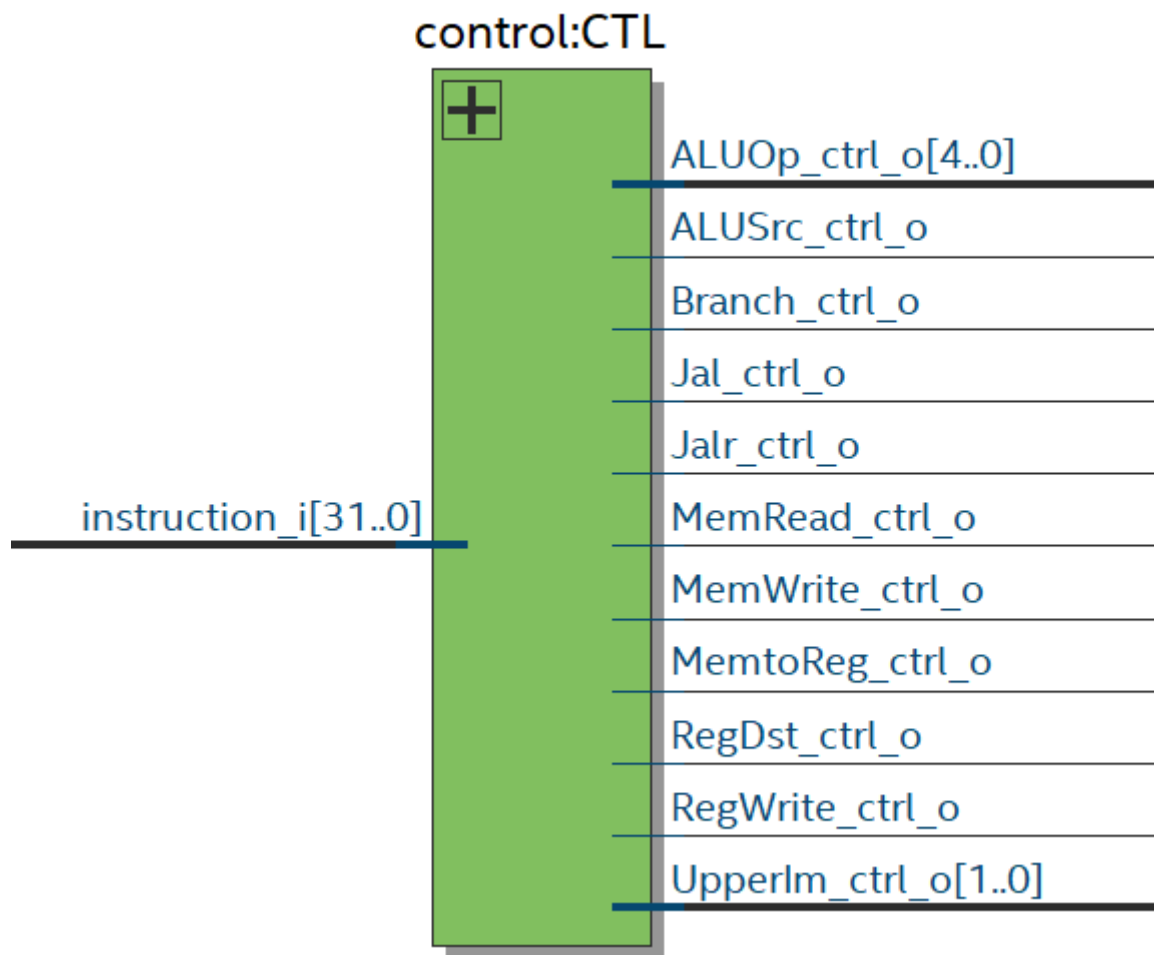


איור 21: RTL Viewer - שלב IF

Idcode:ID

ALUOp_ctrl_i[4..0]	+	ex_ALUOp_ctrl_o[4..0]
ALUSrc_ctrl_i		ex_ALUSrc_ctrl_o
Branch_ctrl_i		ex_Branch_ctrl_o
Jal_ctrl_i		ex_Jal_ctrl_o
Jalr_ctrl_i		ex_Jalr_ctrl_o
MemRead_ctrl_i		ex_MemRead_ctrl_o
MemWrite_ctrl_i		ex_MemWrite_ctrl_o
MemtoReg_ctrl_i		ex_MemtoReg_ctrl_o
RegDst_ctrl_i		ex_RegDst_ctrl_o
RegWrite_ctrl_i		ex_RegWrite_ctrl_o
Upperlm_ctrl_i[1..0]		ex_Upperlm_ctrl_o[1..0]
clk_i		ex_instruction_o[31..0]
flush_i		ex_pc_o[12..0]
instruction_i[31..0]		ex_pc_plus4_o[12..0]
pc_i[12..0]		ex_rd_o[4..0]
pc_plus4_i[12..0]		ex_read_data1_o[31..0]
rst_i		ex_read_data2_o[31..0]
stall_i		ex_rs1_o[4..0]
wb_RegWrite_ctrl_i		ex_rs2_o[4..0]
wb_rd_i[4..0]		ex_sign_ext_o[31..0]
wb_write_data_i[31..0]		id_rs1_o[4..0]
		id_rs2_o[4..0]

איור 22: RTL Viewer - שלב ID

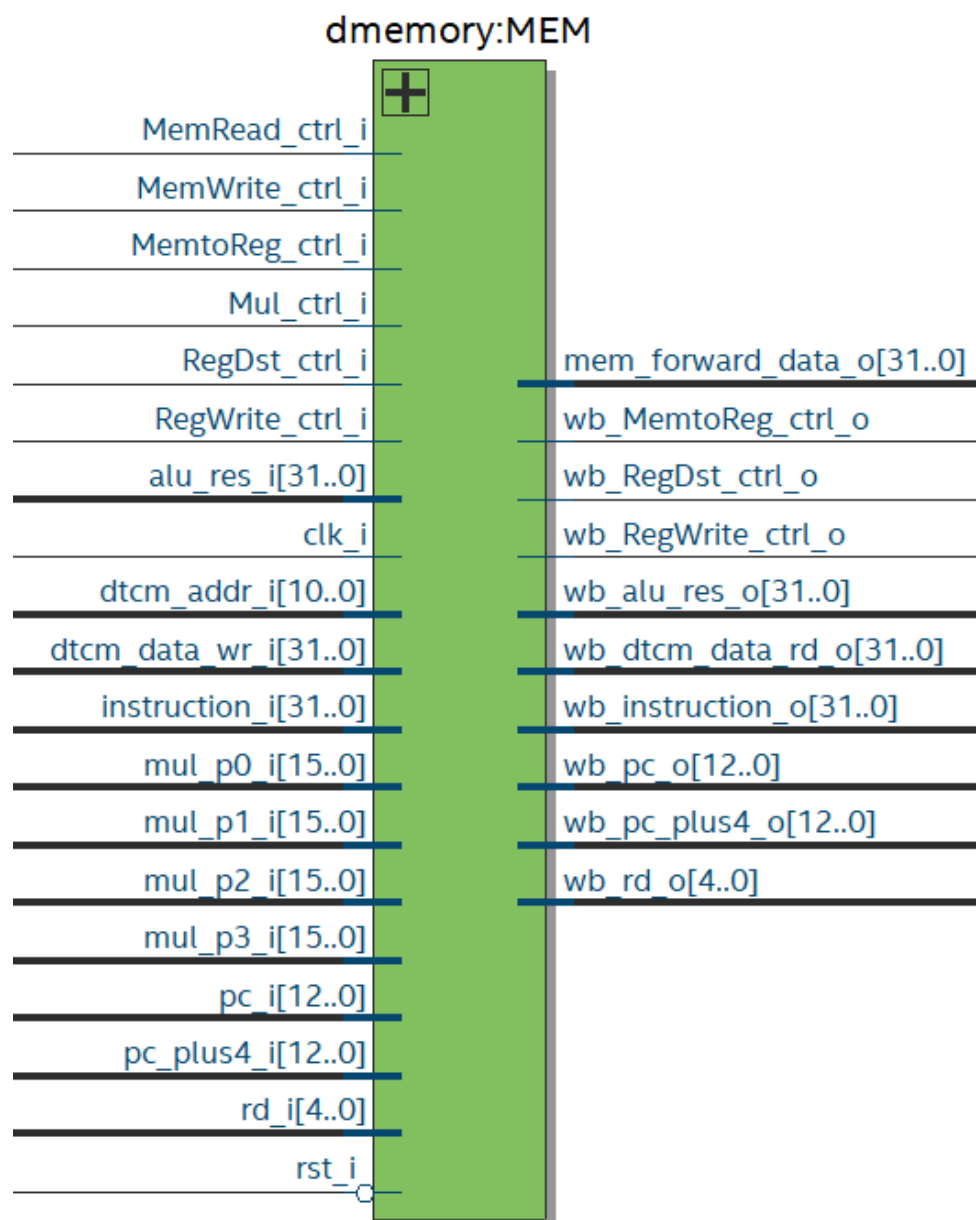


איור 23: RTL Viewer - מודול CONTROL בשלב ID

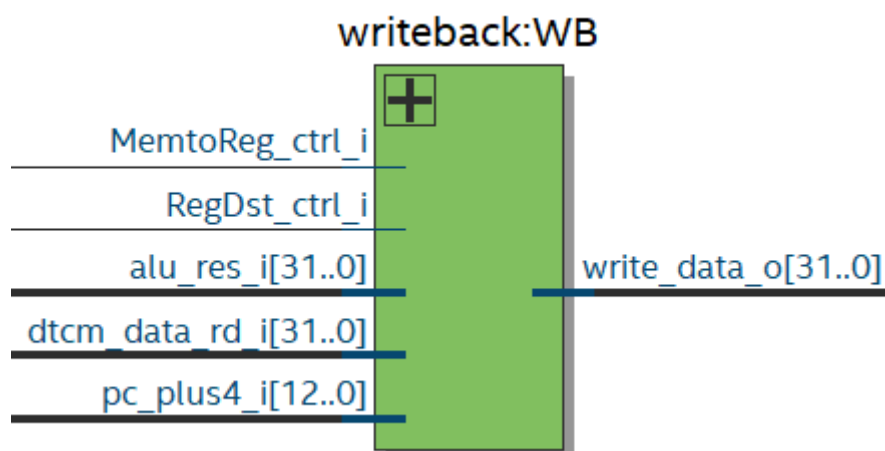
Execute:EXE

ALUOp_ctrl_i[4..0]	+	
ALUSrc_ctrl_i		
Branch_ctrl_i		mem_Branch_ctrl_o
Jal_ctrl_i		mem_Jal_ctrl_o
Jalr_ctrl_i		mem_Jalr_ctrl_o
MemRead_ctrl_i		mem_MemRead_ctrl_o
MemWrite_ctrl_i		mem_MemWrite_ctrl_o
MemtoReg_ctrl_i		mem_MemtoReg_ctrl_o
RegDst_ctrl_i		mem_Mul_ctrl_o
RegWrite_ctrl_i		mem_RegDst_ctrl_o
UpperIm_ctrl_i[1..0]		mem_RegWrite_ctrl_o
clk_i		mem_addr_gen_o[12..0]
flush_i		mem_alu_res_o[31..0]
forward_a_i[1..0]		mem_brTaken_o
forward_b_i[1..0]		mem_instruction_o[31..0]
instruction_i[31..0]		mem_mul_p0_o[15..0]
mem_forward_data_i[31..0]		mem_mul_p1_o[15..0]
pc_i[12..0]		mem_mul_p2_o[15..0]
pc_plus4_i[12..0]		mem_mul_p3_o[15..0]
rd_i[4..0]		mem_pc_o[12..0]
read_data1_i[31..0]		mem_pc_plus4_o[12..0]
read_data2_i[31..0]		mem_rd_o[4..0]
rst_i		mem_write_data_o[31..0]
sign_extend_i[31..0]		
wb_write_data_i[31..0]		

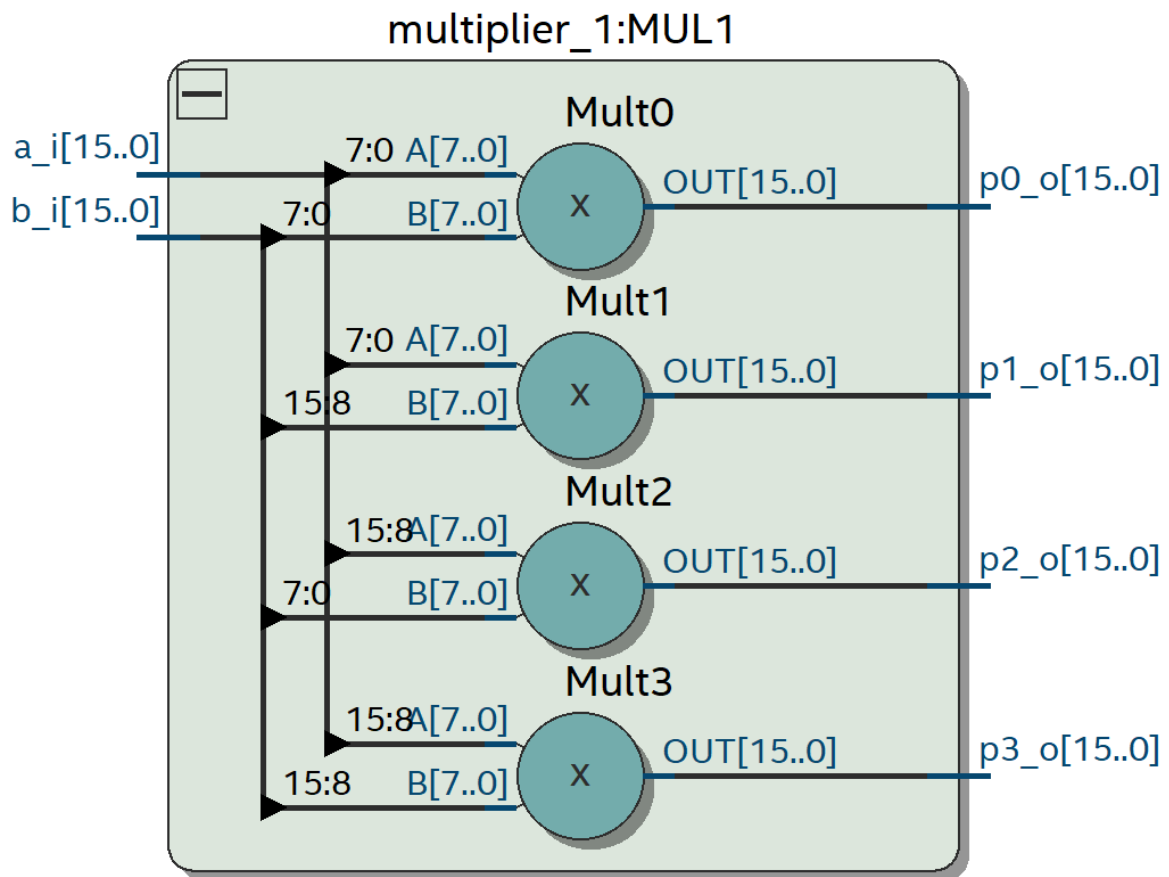
איור 24: RTL Viewer - שלב EX



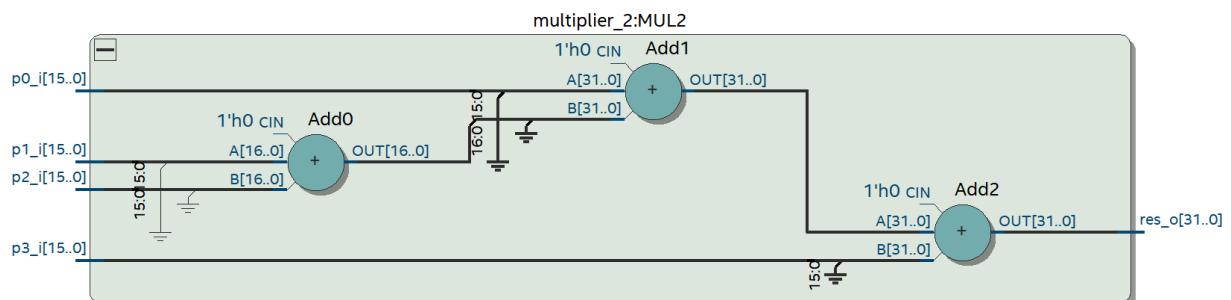
איור 25: RTL Viewer - שלב MEM



איור 26: RTL Viewer - שלב WB



איור 27: multiplier_1:MUL1 – RTL Viewer (שלב EX): ארבעת המכפלים החלקיים 8×8



איור 25: multiplier_2:MUL2 – RTL Viewer (שלב MEM)

5. וריפיקציה ווילדציה של התכן

5.1 מתודולוגיית האימות

האימות בוצע לפי זרימת התכן שבסעיף 8 של המשימה, עם RARS כמודל זהב. עבור כל אפליקציית benchmark, קובץ ה-DTCM.h שמייצר RARS בסוף הריצה משמש כתוצאה הנכונה, ומולו מושווה קובץ ה-DTCM.mem שנשפך מהסימולציה ב-ModelSim, וכן קובץ ה-DTCM.hex שמפיק ISMCE מהריצה על ה-FPGA.

אפליקציות ה-benchmark מסתיימות בלולאה אינסופית (הוראת `beq x0,x0,0` בקידוד 00000063). במימוש ה-single-cycle מספיק לזהות הוראה זו בשלב הפענוח כדי לעצור, אבל ב-pipeline זיהוי כזה שגוי: ההסתעפויות מוכרעות ב-MEM, ולכן השליפה הספקולטיבית מביאה את הוראת הסיום לשלב ID כבר באיטרציה הראשונה של לולאה כלשהי, ועצירה על כך הייתה מפסיקה את הריצה מוקדם מדי. לכן נעשה שימוש בקריטריון אחר: אירוע flush שיעד ההפניה שלו שווה ל-PC+4 של ההוראה המפנה עצמה (`target+4 == mem_pc_plus4`). תנאי כזה יכול להתקיים רק עבור קפיצה עצמית, ולכן הוא חד-משמעי. במימוש ה-single-cycle העצירה נעשית פשוט על ההוראה המפוענחת 00000063.

5.2 תוצאת ההשוואה מול מודל הזהב

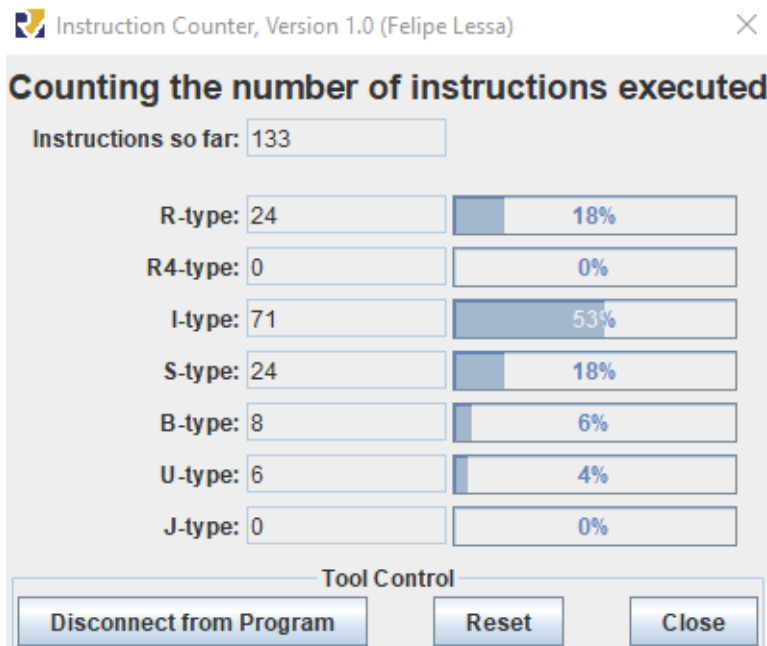
כל ארבע האפליקציות הורצו בשני המימושים. תמונות זיכרון הנתונים שנשפכו נבדקו הן מול DTCM.h של RARS והן זו מול זו.

ההשוואה הישירה בין DTCM_testN.mem של ה-single-cycle לבין זה של ה-pipeline העלתה זהות מוחלטת בכל 1024 המילים ובכל ארבעת הטסטים. מכאן ששני המימושים שקולים פונקציונלית: מנגנוני ה-pipeline – forwarding, interlock ל-load-use, ביטול הוראות ספקולטיביות והכרעת הסתעפויות ב-MEM, וכן פיצול הכפל לשני שלבים – אינם משנים ולו ביט אחד בתוצאה הסופית של התוכנית.

5.3 ניתוח דיאגרמות גלים

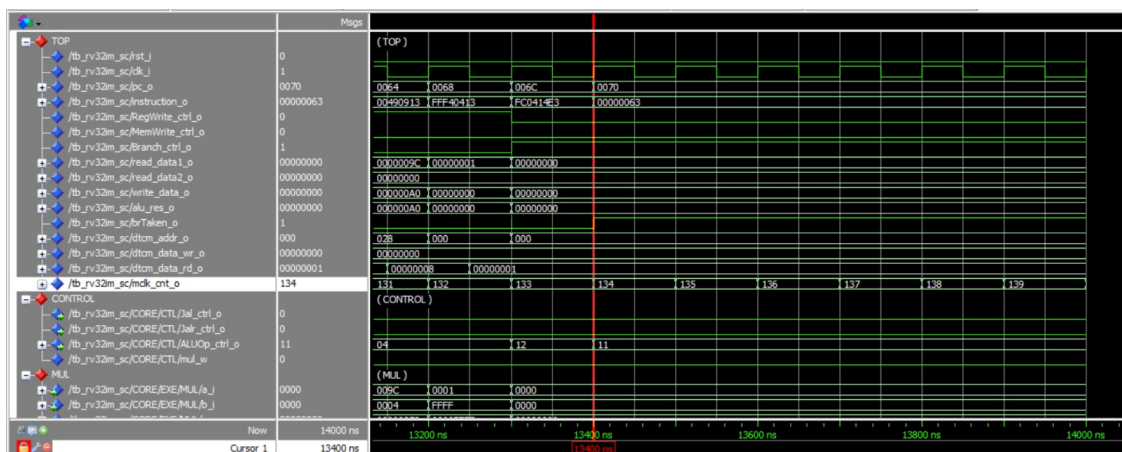
מחזור השעון ב-testbench הוא 100 ns, ולכן זמן הסימולציה מתורגם ישירות למחזורים. על בסיס זה ניתחנו את דיאגרמות הגלים של ארבע האפליקציות בשני המימושים ואת התנהגות התזמון של המערכת. לכל בדיקה תועדו: תוצאת מונה ההוראות של RARS, תחילת הריצה ב-ModelSim ובחומרה (Signal-Tap על ה-FPGA) עבור שני המימושים, וסוף הריצה.

5.3.1 test1:



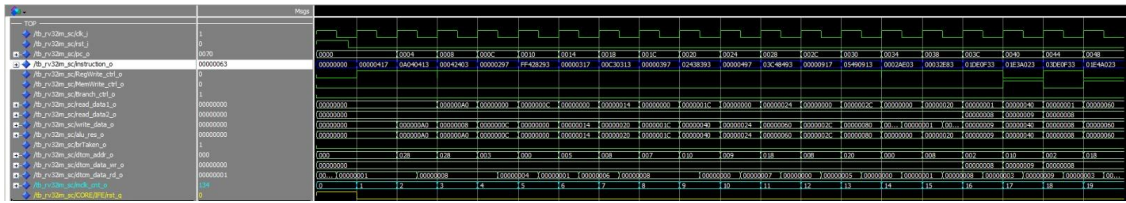
איור 27: RARS Instruction Counter עבור test1

Single Cycle

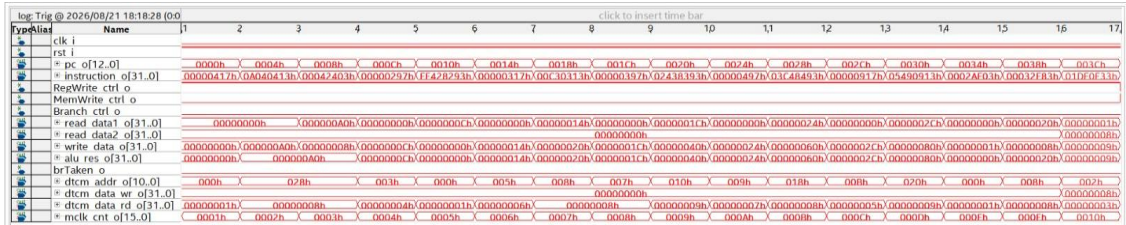


איור 28: test1 (Single Cycle), סוף הריצה

מספר המחזורים (134) שווה למספר ההוראות שבוצעו, כצפוי מ- $IPC=1$ (גדול באחד ממונה RARS, שאינו סופר את הוראת הסיום עצמה).



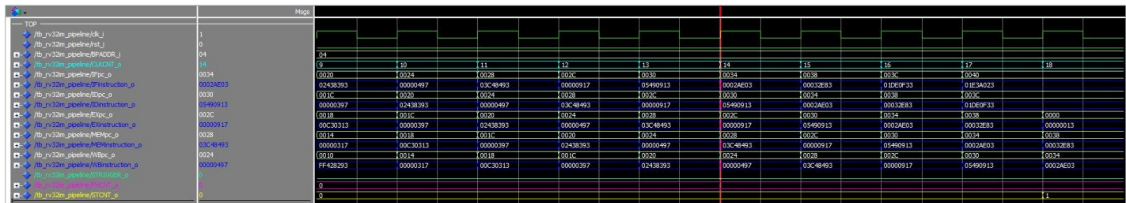
איור 29: ModelSim (Single Cycle) test1, תחילת ריצה ב-



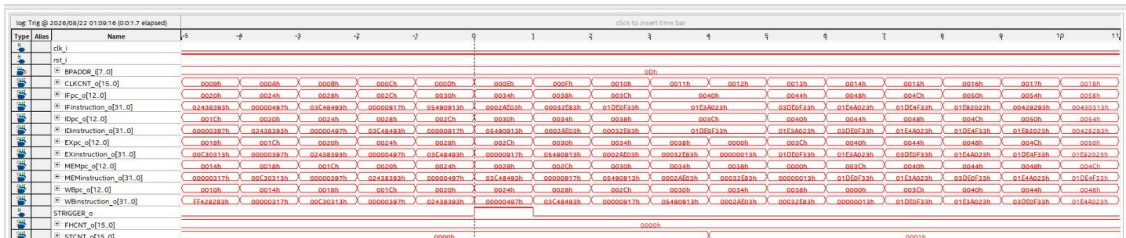
איור 30: ModelSim (Single Cycle) test1, אותה תחילת ריצה כפי שנצפתה ב-Signal-Tap

אותו רצף הקסדימלי של הוראות כמו ב-ModelSim, אימות נוסף לכך שהתכן שסונתז זהה לזה שנבדק בסימולציה

Pipeline



איור 31: ModelSim (Pipeline) test1, תחילת ריצה ב-



איור 32: ModelSim (Pipeline) test1, אותה תחילת ריצה כפי שנצפתה ב-Signal-Tap

אותו רצף הקסדימלי של הוראות כמו ב-ModelSim, אימות נוסף לכך שהתכן שסונתז זהה לזה שנבדק בסימולציה

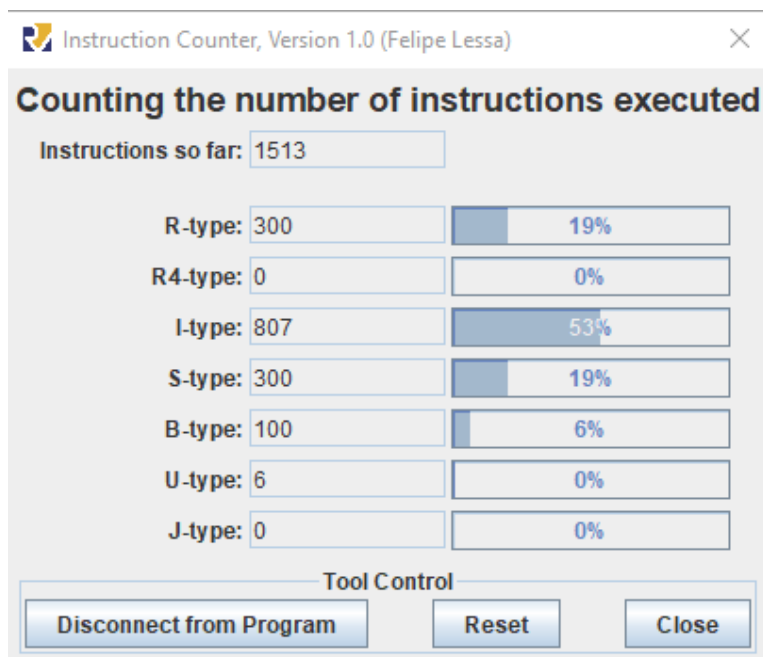
MUL_STAGE1		(MUL_STAGE1)			
/tb_rv32m_pipeline/CORE/EXE/MUL1/a_i	0140	0780	0140	02D0	0460
/tb_rv32m_pipeline/CORE/EXE/MUL1/b_i	0004	0000	0004		
/tb_rv32m_pipeline/CORE/EXE/MUL1/p0_o	0100	0000	0100	0340	0180
/tb_rv32m_pipeline/CORE/EXE/MUL1/p1_o	0000	0000			
/tb_rv32m_pipeline/CORE/EXE/MUL1/p2_o	0004	0000	0004	0008	0010
/tb_rv32m_pipeline/CORE/EXE/MUL1/p3_o	0000	0000			
MUL_STAGE2		(MUL_STAGE2)			
/tb_rv32m_pipeline/CORE/MEM/mul_res_w	00000000	00000690	00000000	00000500	00000B40
/tb_rv32m_pipeline/CORE/MEM/mem_result_w	00000780	00000038	00000780	00000144	000002D4

איור 33: ModelSim (Pipeline) test1, פעולת הכופל המפוצל ב-ModelSim

$a_i=0140, b_i=0004$ מניבים $p0_o=0100, p2_o=0004, p1_o=p3_o=0000$;

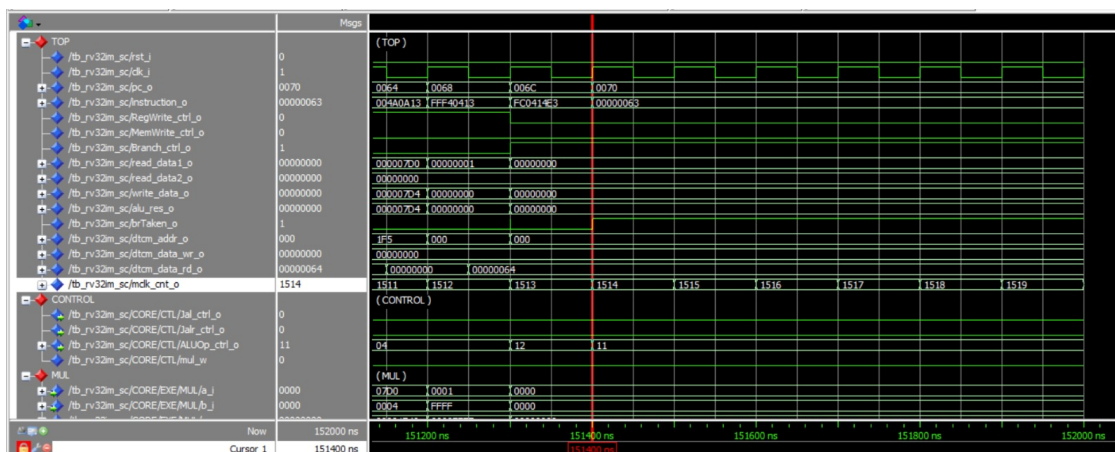
במחזור הבא, $mul_res_w = 0x140 \times 4 = 00000500$ — ההוכחה הישירה לכך שהכפל אכן מתפרש על שני מחזורי pipeline נפרדים

test2 5.3.2:



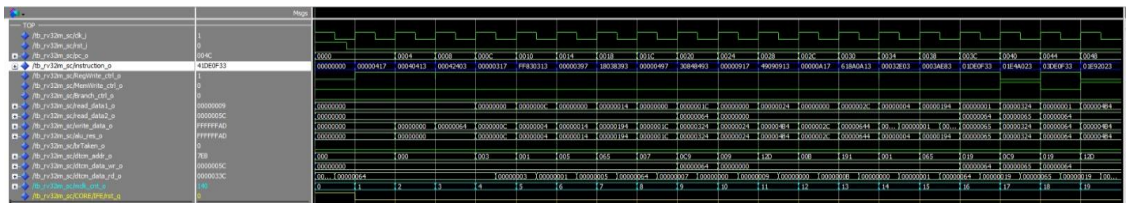
איור 35: RARS Instruction Counter עבור test2

Single Cycle



איור 38: test2 (Single Cycle), סוף הריצה

מספר המחזורים (1514) שווה למספר ההוראות שבוצעו, כצפוי מ- $IPC=1$ (גדול באחד ממונה RARS, שאינו סופר את הוראת הסיום עצמה).



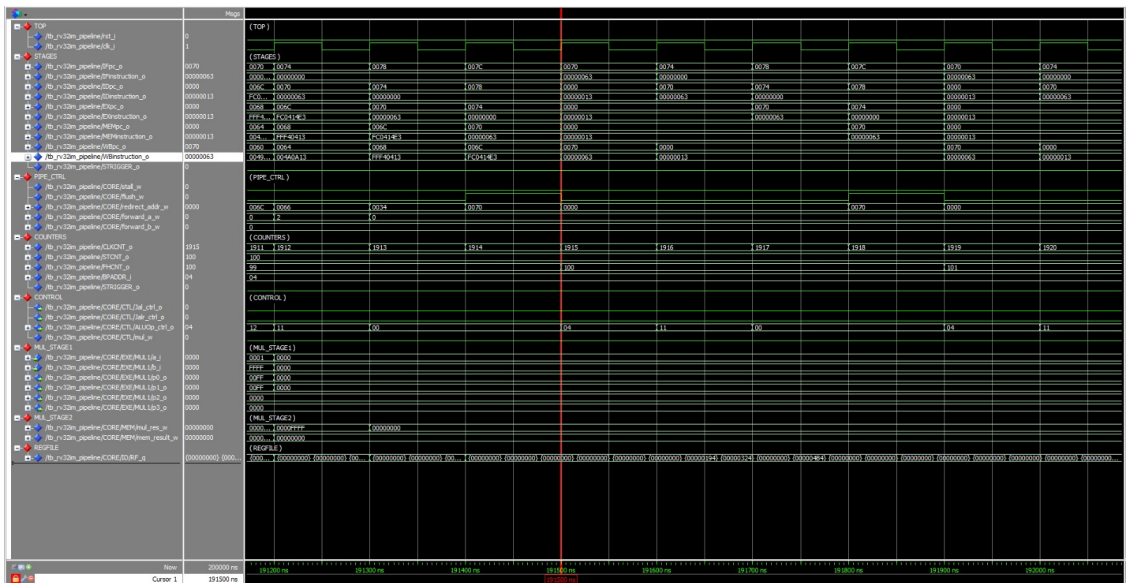
איור 36: test2 (Single Cycle), תחילת ריצה



איור 37: test2 (Single Cycle), תחילת ריצה כפי שנצפתה ב-Signal-Tap

אותו רצף הקסדצימלי של הוראות כמו ב-ModelSim, אימות נוסף לכך שהתכן שסונתז זהה לזה שנבדק בסימולציה

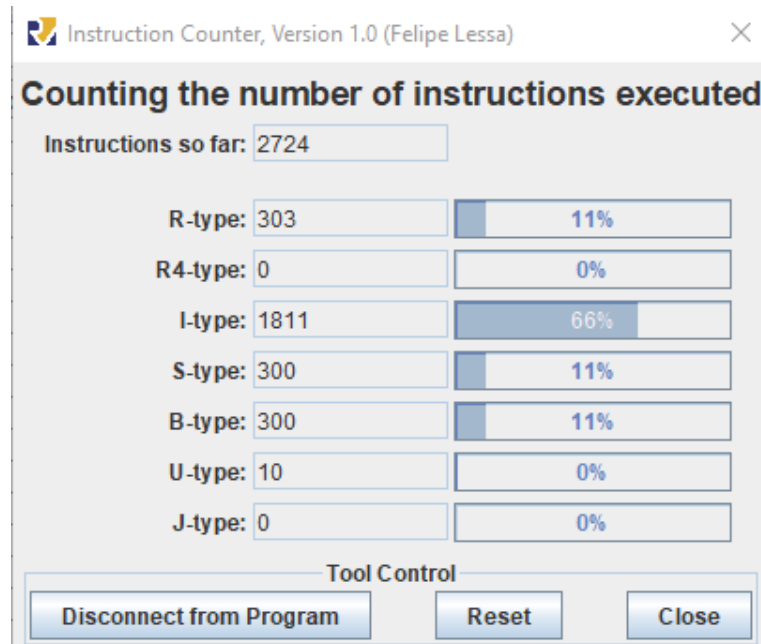
Pipeline



איור 41: test2 (Pipeline), סוף הריצה

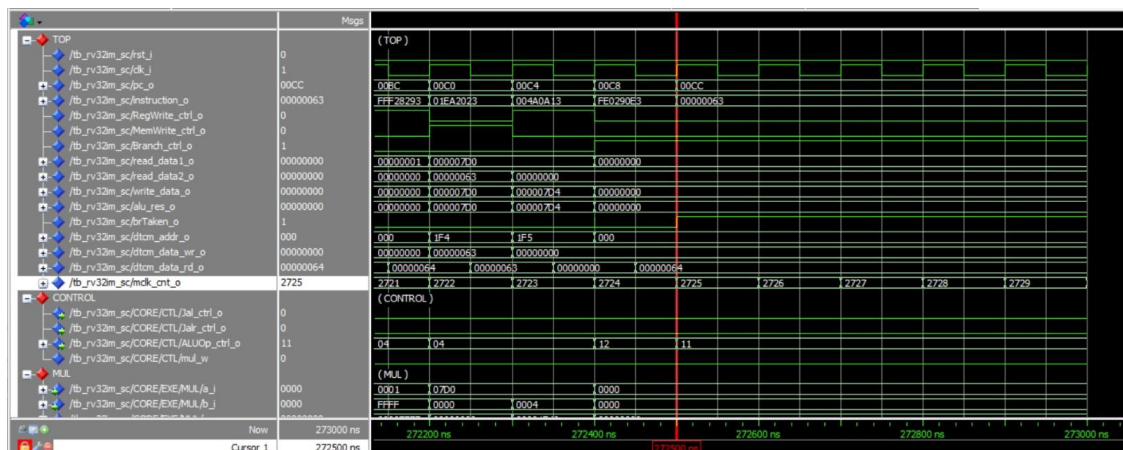
CLKCNT_0 גדול ממספר ההוראות בשל תקורת ה-flush/stalls; ההשוואה המלאה מול ה-IPC התיאורטי מופיעה בטבלה 15 שבסעיף 7.

5.3.3 test3: לולאות והסתעפויות נלקחות



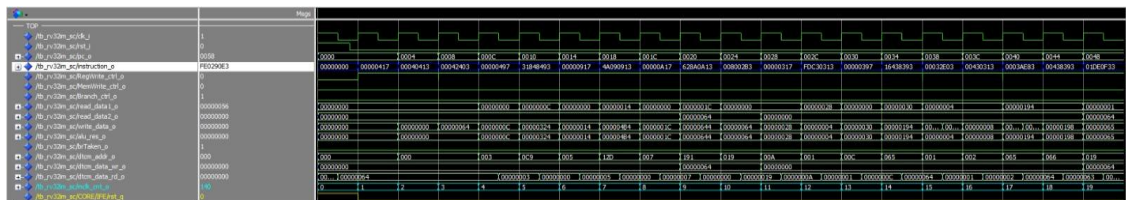
איור 42: RARS Instruction Counter עבור test3

Single Cycle

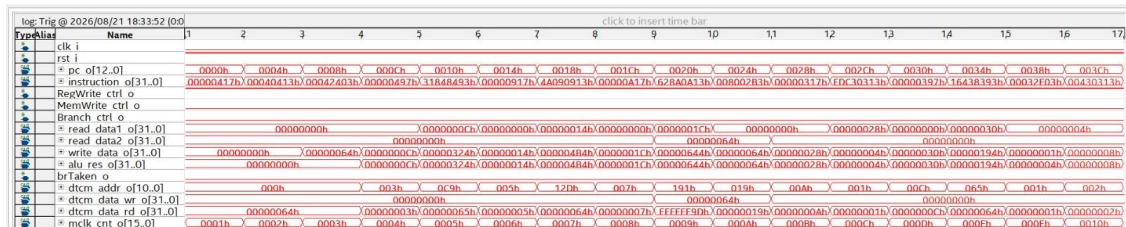


איור 45: test3 (Single Cycle), סוף הריצה

מספר המחזורים (2725) שווה למספר ההוראות שבוצעו, כצפוי מ- $IPC=1$ (גדול באחד ממונה RARS, שאינו סופר את הוראת הסיום עצמה).



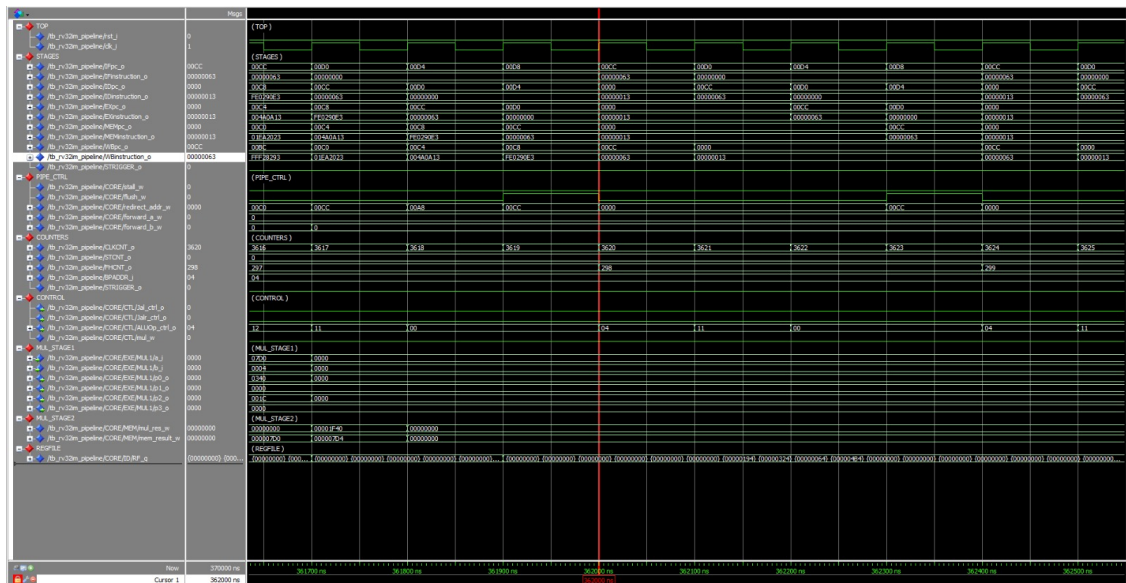
איור 43: test3 (Single Cycle), תחילת ריצה



איור 44: test3 (Single Cycle), תחילת ריצה כפי שנצפתה ב-Signal-Tap

אותו רצף הקסדיצימלי של הוראות כמו ב-ModelSim, אימות נוסף לכך שהתכן שסונתז זהה לזה שנבדק בסימולציה

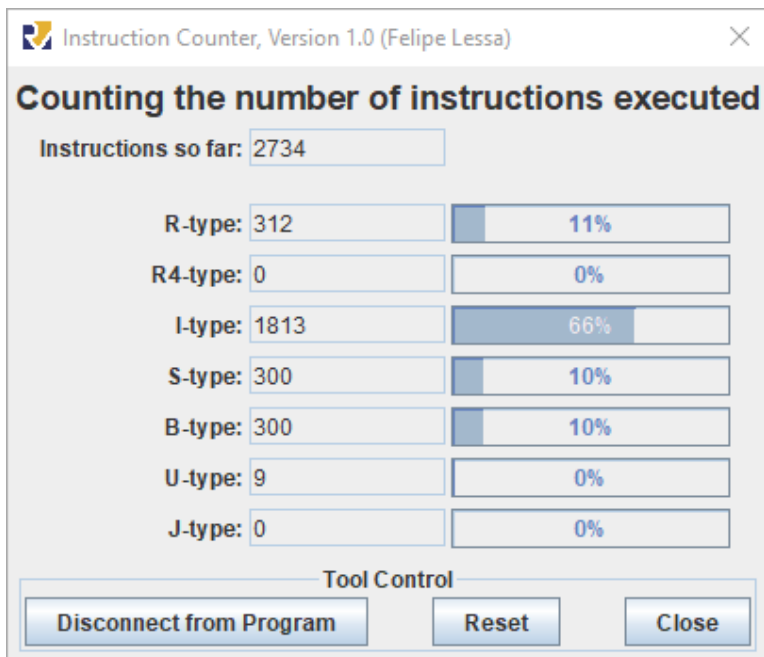
Pipeline



איור 48: test3 (Pipeline), סוף הריצה

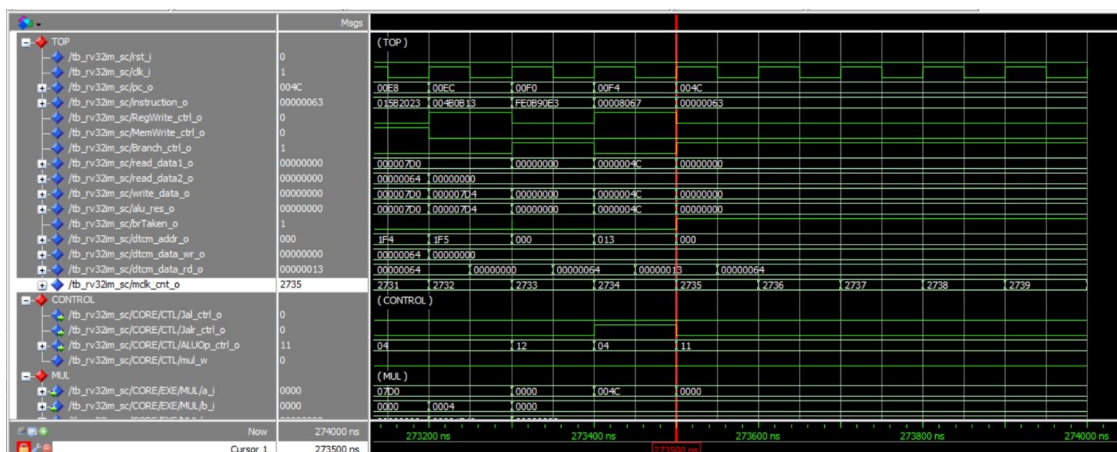
CLKCNT_0 גדול ממספר ההוראות בשל תקורת ה-flush/stalls; ההשוואה המלאה מול ה-IPC התיאורטי מופיעה בטבלה 15 שבסעיף 7.

test4 5.3.4: קפיצות בלתי-מוותנות וקריאות לפונקציות



איור 49: RARS Instruction Counter עבור test4

Single Cycle



איור 52: test4 (Single Cycle), סוף הריצה

מספר המחזורים (2735) שווה למספר ההוראות שבוצעו, כצפוי מ- $IPC=1$ (גדול באחד ממונה RARS, שאינו סופר את הוראת הסיום עצמה).

6. ניתוח PPA

6.1 שלוש טבלאות PPA-

שלוש טבלאות אפיון ה-PPA הנדרשות בסעיף 7 של הגדרת המשימה מרוכזות כאן יחד. צילומי דוחות ה-Quartus שמהם נלקחו הערכים, והניתוח המפורט של כל מדד, מופיעים בסעיפים 6.2–6.4.

Area						
	Logic elements	Registers	I/O pins	Embedded memory bits	Embedded 9-bit Multipliers	PLLs
Single-Cycle RV32IM	3,120	1,279	270	131,072	4	1
Pipelined RV32IM	3,538	1,877	284	131,072	4	1

טבלה 10: צריכת המשאבים של שני המימושים

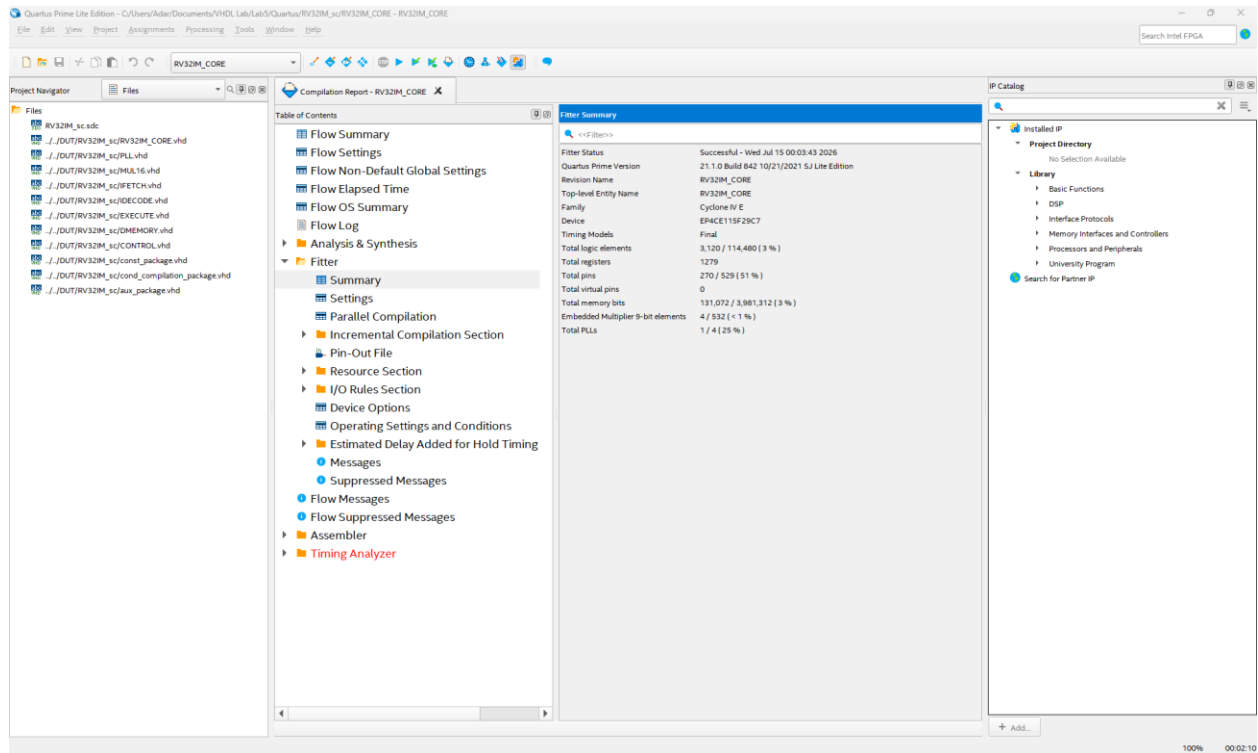
Performance			
	$f_{\max}$	Critical path (what is the slowest submodule and why does it cause the critical path)	$f_{\text{MCLK}} (= f_{\text{sysclk}})$
Single-Cycle RV32IM	26.81 MHz	המסלול של sw/lw: מה-ITCM, דרך מרבבי ה-IDECODE וה-ALU, עד אוגר הכתובת של ה-DTCM. הכתובת היא תוצאת ה-ALU, ולכן כל השרשרת חייבת להסתיים לפני הגישה לזיכרון.	25 MHz
Pipelined RV32IM	51.7 MHz	המסלול של forwarding ב-EX: מאוגר EX/MEM (ex_mem_instruction_q[7]) שהוא rd[0] ב-MEM), דרך יחידת ה-forwarding, מרבבי האופרנדים וה-ALU, עד ex_mem_alu_res_q[15]. תוצאת ה-MEM מוזרמת ל-EX באותו מחזור, ולכן כל השרשרת חייבת להסתיים לפני דפדוף EX/MEM.	25 MHz

טבלה 11: תדר מרבי, מסלול קריטי ותדר העבודה בפועל

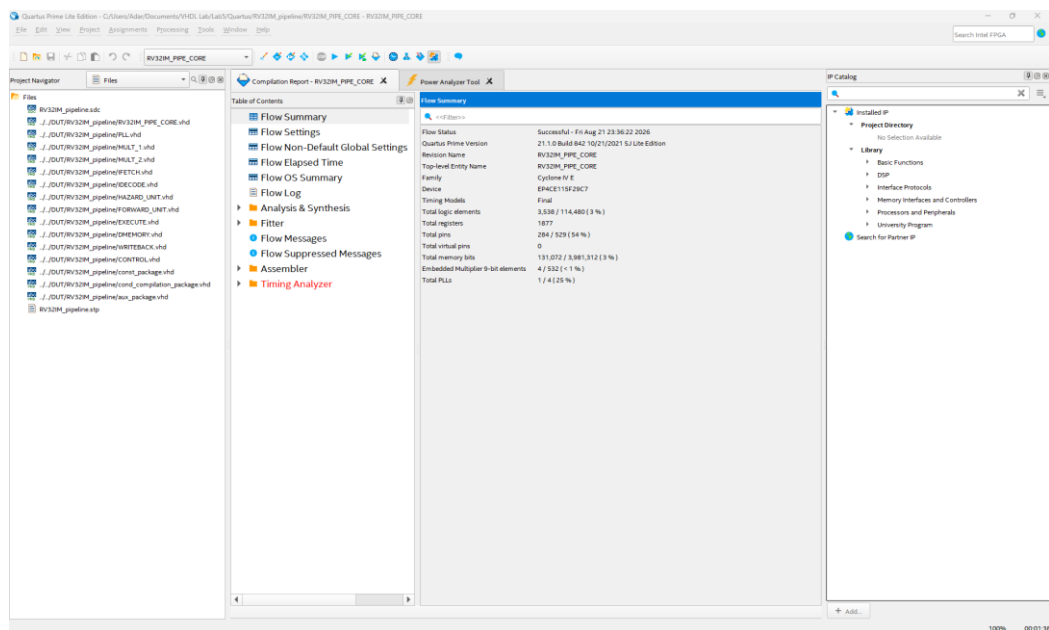
Power				
	Total power consumption	Static power consumption	Dynamic power consumption	I/O power consumption
Single-Cycle RV32IM	275.98 mW	102.39 mW	22.93 mW	150.67 mW
Pipelined RV32IM	292.21 mW	102.49 mW	31.81 mW	157.91 mW

טבלה 12: צריכת ההספק של שני המימושים

Area 6.2: צריכת משאבים



איור 42: דוח Fitter Summary של RV32IM_CORE במימוש ה-single-cycle



איור 43: דוח Fitter Summary של RV32IM_PIPE_CORE במימוש ה-pipeline

ניתוח

- מספר האוגרים גדל ל-1,877 (+46.8% לעומת ה-Single-Cycle), הגידול הגדול ביותר מבין המדדים וזהו המחיר הישיר של ה-pipeline. ארבעה אוגרי pipeline (IF/ID, ID/EX, EX/MEM, MEM/WB) חייבים לשמור בכל מחזור את ההוראה, ה-PC, שני האופרנדים, ה-immediate, מזהי האוגרים וכל אותות הבקרה, וכעת גם ארבע תוצאות ביניים בנות 16 ביט של הכפל (P0–P3) באוגר EX/MEM. נוספים להם שלושת מוני הניפוי בני 16 הביט ואוגר BPADDR.
- הלוגיקה גדלה ל-3,538 Logic elements – עלייה של 13.4% בלבד, נמוכה משמעותית מהגידול באוגרים. מסלול הנתונים עצמו לא הוכפל; נוספו FORWARD_UNIT, HAZARD_UNIT, מרבבי ה-forwarding, לוגיקת ה-redirect, מודול WRITEBACK ומחבר הצירוף ש-B2_MULT.
- כמו ב-Single-Cycle, נוצלו 4 כופלים משובצים – פיצול המכפל היא לוגיקת חיבור בלבד, ואינה צורכת כופלים נוספים.
- הגידול של +14 פינים (270→284) נובע מאותות הניפוי של CLKCNT_o[15:0], Figure 8: STCNT_o[15:0], FHCNT_o[15:0], BPADDR_i[7:0], STRIGGER_o ו-flush_w-PC/instruction (IFpc_o...WBinstruction_o). stall_w יוצאים כפינים עליונים.

Performance 6.3: תדר מרבי ומסלול קריטי

Timing Analyzer - C:\Users\Adar\Documents\VHDL Lab\Lab5\Quartus\RV32M_core\RV32M_CORE - RV32M_CORE

Set Operating Conditions: Slow 1200mV BSC Model

Tasks: Read SDC File, Update Timing Netlist, Reset Design, Set Operating Conditions..., Reports, Slack, Report Setup Summary, Report Hold Summary, Report Recovery Summary, Report Removal Summary, Report Minimum Pulse Width Summary, Report Max Skew Summary, Report Net Delay Summary, Dashboard, Report Fmax Summary, Report Datasheet, Device Specific, Report TCCS, Report RSKM.

Report: Timing Analyzer Summary, Advanced I/O Timing, SDC File List, Fmax Summary, Slow 1200mV BSC Model, Slow 1200mV OC Model, Fast 1200mV OC Model, Summary (Setup).

	Fmax	Restricted Fmax	Clock Name	Note
1	26.81 MHz	26.81 MHz	\G0\MCLK[altpll_component][pllclk[0]	
2	73.62 MHz	73.62 MHz	altdev_reserved_clk	

This panel reports FMAX for every clock in the design, regardless of the user-specified clock periods. FMAX is only computed for paths where the source and destination registers or ports are driven by the same clock. Paths of different clocks, including generated clocks, are ignored. For paths between a clock and its inversion, FMAX is computed as if the rising and falling edges are scaled along with FMAX, such that the duty cycle (in terms of a percentage) is maintained. Altera recommends that you always use clock constraints and other slack reports for sign-off analysis.

0% 00:00:00 Ready

איור 44: דוח Fmax Summary של מימוש ה-single-cycle: 26.81 MHz על שעון הליבה MCLK

Timing Analyzer - C:\Users\Adar\Documents\VHDL Lab\Lab5\Quartus\RV32M_pipeline\RV32M_PIPE_CORE - RV32M_PIPE_CORE

Set Operating Conditions: Slow 1200mV BSC Model

Tasks: Report Removal Summary, Report Minimum Pulse Width Summary, Report Max Skew Summary, Report Net Delay Summary, Dashboard, Report Fmax Summary, Report Datasheet, Device Specific, Report TCCS, Report RSKM, Report DDR, Report Metastability Summary, Diagnostics, Report Clocks, Report Clock Time, Report Clock Transfers, Report Unconstrained Paths, Report SDC, Report Ignored Constraints.

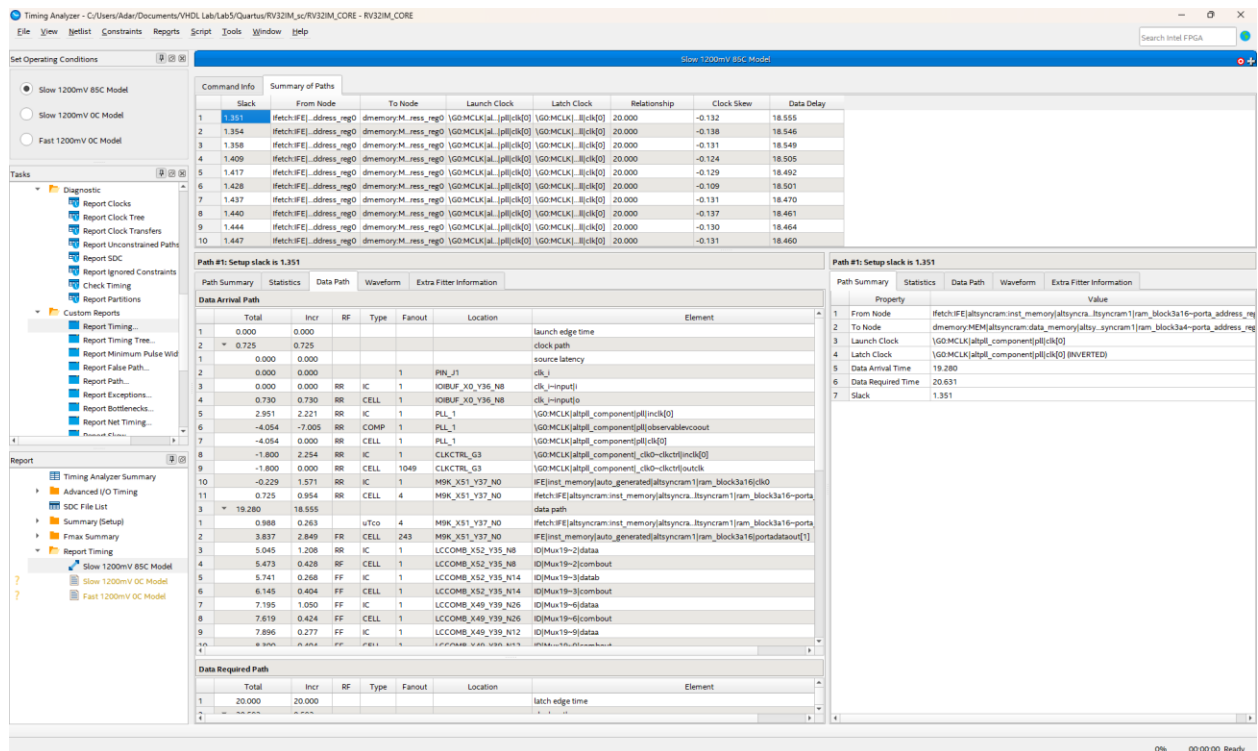
Report: Timing Analyzer Summary, Advanced I/O Timing, SDC File List, Report Timing, Fmax Summary, Slow 1200mV BSC Model, Slow 1200mV OC Model, Fast 1200mV OC Model.

	Fmax	Restricted Fmax	Clock Name	Note
1	51.7 MHz	51.7 MHz	\G0\MCLK[altpll_component][pllclk[0]	
2	90.27 MHz	90.27 MHz	altdev_reserved_clk	

This panel reports FMAX for every clock in the design, regardless of the user-specified clock periods. FMAX is only computed for paths where the source and destination registers or ports are driven by the same clock. Paths of different clocks, including generated clocks, are ignored. For paths between a clock and its inversion, FMAX is computed as if the rising and falling edges are scaled along with FMAX, such that the duty cycle (in terms of a percentage) is maintained. Altera recommends that you always use clock constraints and other slack reports for sign-off analysis.

0% 00:00:00 Ready

איור 45: דוח Fmax Summary של מימוש ה-pipeline: 51.7 MHz על שעון הליבה MCLK



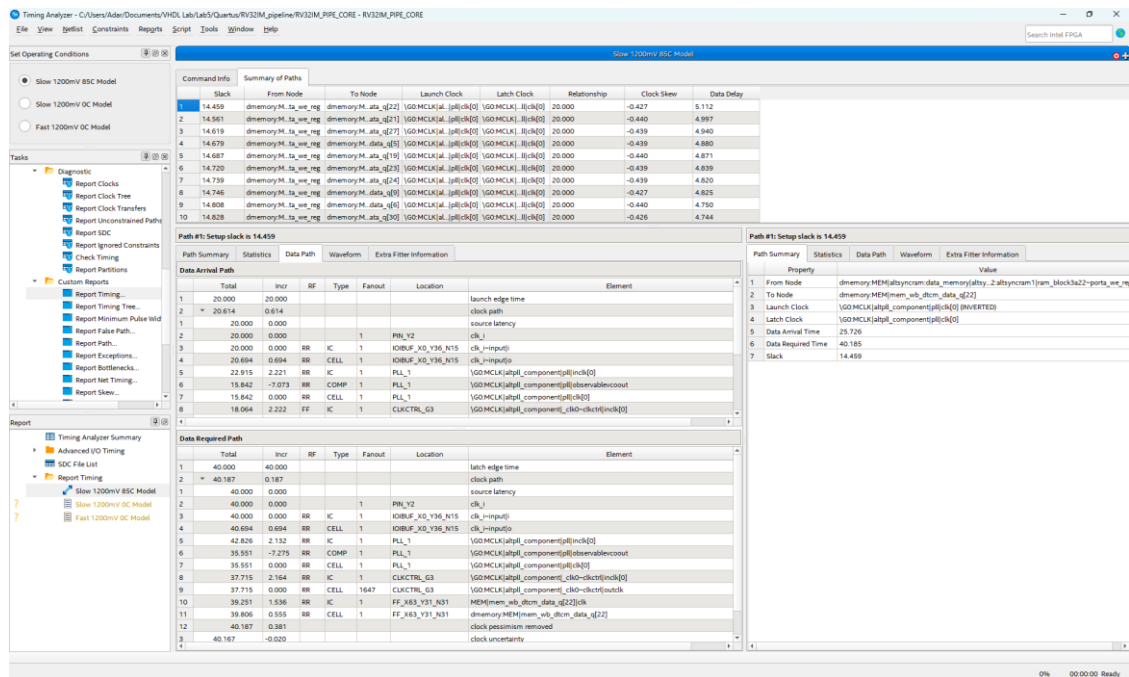
איור 46: המסלול הקריטי של מימוש ה-single-cycle, מ-`ifetch:IFE|...|ram_block|porta_address_reg`, `dmemory:MEM|...|porta_address_reg` Latch Clock מסומן INVERTED, ולכן זהו מסלול חצי-מחזור

חישוב ה- f_{max} מתוך המסלול

השהיית המסלול היא 18.555 ns בלבד, ולכאורה הייתה מאפשרת ≈ 53 MHz, אלא שכפי שהוסבר בסעיף 3.1, ה-DTCM נכתב על השעון ההפוך, וזה נראה מפורשות בשדה Latch Clock שבדוח, המסומן INVERTED. לכן המסלול כולו חייב להסתיים בתוך חצי מחזור שעון:

$$f_{max} = 1 / (2 \cdot t_{path}) = 1 / (2 \cdot 18.65 \text{ ns}) \approx 26.8 \text{ MHz}$$

החישוב מתאים ל-26.81 MHz שדיווח ה-Timing Analyzer, ומכאן שזיהינו נכון את המסלול.



איור 47: דוח Report Timing של ה-pipeline: עשרת המסלולים בעלי ה-slack הקטן ביותר: כולם מסלולי DTCM (mem_wb_dtcn_data_q) מ-memory:MEM אל ram_block; slack מקסימלי בטבלה זו 14.459

זיהוי המסלול הקריטי של ה-pipeline

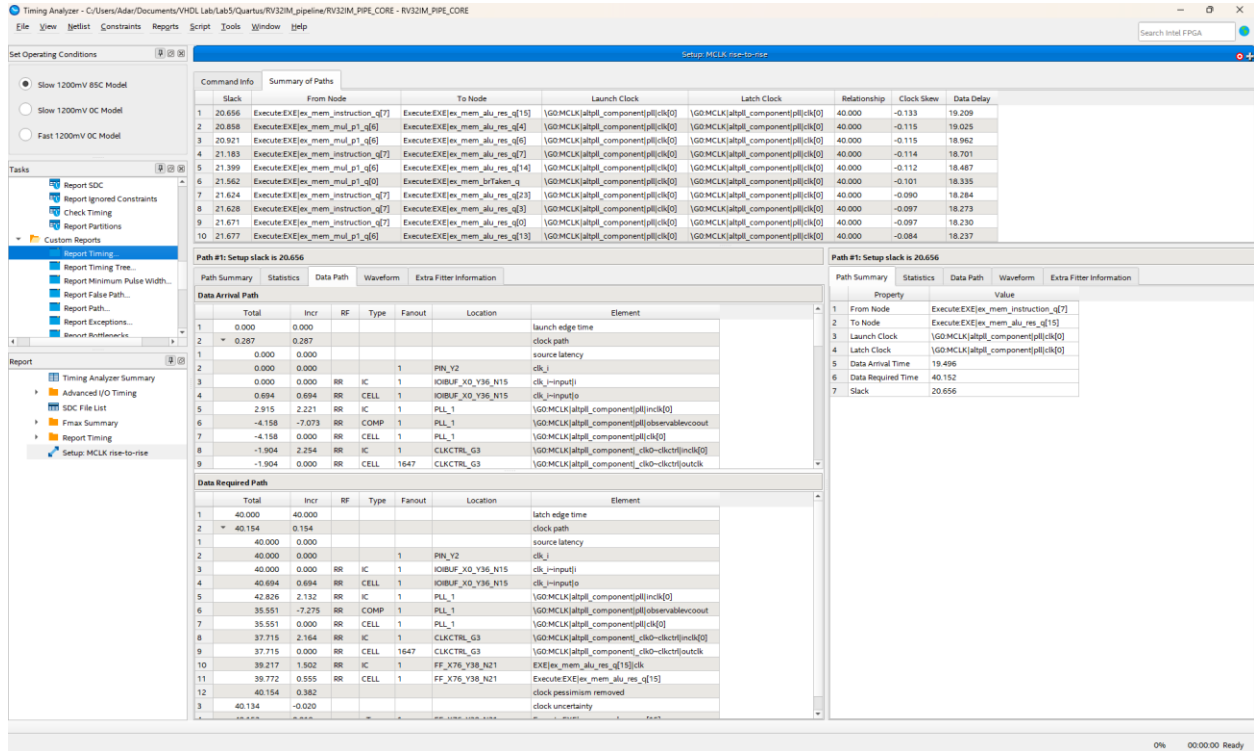
זיהוי המסלול הקובע את fmax ב-pipeline: כמו ב-Single-Cycle, ה-DTCM ב-pipeline עדיין נכתבת על השעון ההפוך (DMEMORY.vhd, wrclk_w <= NOT clk_i), כך שבתכן מתקיימים בריזומנית מסלולי חצי-מחזור (לתוך אוגרי ה-DTCM) ומסלולי מחזור-מלא (בין אוגרי ה-pipeline). דרישת הזמן של הראשונים משתנה כמו T/2 ולא האחרונים כמו T, ולכן ה-slack שמדווח Report Timing עבור שני הסוגים אינו בר-השוואה ישירה. עשרת המסלולים המוצגים באיור 33 הם כולם מסוג DTCM (slack עד 14.459 ns, השהיית נתונים טהורה של 5.112 ns בלבד עבור המסלול הראשון), כלומר $f_{max_DTCM} \approx 1/(2 \cdot 5.112 \text{ ns}) \approx 97.8 \text{ MHz}$ — הרבה מעל 51.7 MHz. משמעות הדבר: המסלול המדווח כבעל ה-slack הקטן ביותר אינו המסלול שקובע בפועל את 51.7 MHz.

סוג המסלול	השהיה	תדר מרבי שהמסלול מתיר
EX/MEM → כתובת ה-DTCM (חצי-מחזור, מהדוח)	5.112 ns	$1 / (2 \cdot 5.112) \approx 97.8 \text{ MHz}$
המסלול הקובע בפועל (מחזור-מלא)	19.34 ns	$1 / 19.34 \approx 51.7 \text{ MHz}$

טבלה 14: השהיית מסלול חצי-מחזור ומסלול המחזור-מלא ב-pipeline והתדר שכל אחד מהם מתיר; מכאן שהמסלול בעל ה-slack הקטן ביותר אינו זה שקובע את fmax

כלומר fmax אינו מוגבל עוד על ידי ה-DTCM (כפי שהיה ב-Single-Cycle) אלא על ידי מסלול מחזור-מלא כלשהו בין שני אוגרי pipeline סמוכים, שאורכו כ-19.34 ns. הדוח שברשותנו אינו חושף במפורש את זהות המסלול הזה (עשרת המסלולים הגרועים ביותר התבררו כשייכים ליחס השעון החצי-מחזורי, שלו slack חיובי גדול יותר באופן יחסי לתקופתו הקצרה יותר). מבחינה מבנית, המועמד הסביר ביותר להיות שלב ה-EX/ID: שלב ה-ID עדיין מרכז את קריאת קובץ האוגרים 32×32, את הפענוח המלא ב-CONTROL, את ייצור חמש תבניות ה-immediate ואת בדיקת ה-load-use ב-HAZARD_UNIT – בדיוק כפי שהיה קודם. לעומת זאת שלב ה-EX נעשה קליל יותר משמעותית ביחס למה שהיה ב-Single-Cycle: הוא מכיל כעת רק את ה-ALU ואת ארבעת המכפלים החלקיים המהירים של MULT_1 (מבוססי כופל חומרתי), ולא עוד את שרשרת הסכימה המלאה של MUL16. זהו כיוון הסבר מבני סביר לתוצאת ה-fmax היחסית הגבוהה שהתקבלה (51.7 MHz): פיצול הכפל בין MULT_1 ל-MULT_2 הקל את שלב ה-EX ביחס למצב שבו כל שרשרת הכפל הייתה יושבת בשלב יחיד.

זיהוי ישיר של המסלול הקריטי בפועל של ה-pipeline



איור 48: דוח Report Timing: המסלול הראשון והגרוע ביותר (Slack=20.656 ns) הוא
 Execute:EX|ex_mem_instruction_q[7] → Execute:EX|ex_mem_alu_res_q[15]

נתוני המסלול הקובע, כפי שדווחו על ידי הכלי: Data Delay = 19.209 ns, Data Arrival Time = 19.496 ns
 ו-Slack = 20.656 ns Data Required Time = 40.152 ns.

מכאן שהתדר המרבי שהמסלול הזה מתיר הוא $f_{max} = 1/19.344 \text{ ns} \approx 51.7 \text{ MHz}$ – התאמה מדויקת לערך שדווח
 דוח ה-Fmax Summary שהוצג קודם בסעיף זה, המאשרת שזהו אכן המסלול הקובע בפועל את f_{max} של ה-
 pipeline.

שני קצות המסלול (ex_mem_instruction_q[7] ו-ex_mem_alu_res_q[15]) הם שני שדות של אותו אוגר עצמו
 – אוגר ה-EX/MEM. כלומר המסלול הקריטי בפועל אינו חוצה גבול שלם בין שני שלבי pipeline, אלא הוא מסלול
 קומבינטורי הפנימי ללוגיקת שלב ה-EX שמזינה את כניסת ה-D של אוגר ה-EX/MEM (ככל הנראה שרשרת
 הבחירה/מרבוב בין תוצאת ה-ALU לתוצאת MULT_1, המושפעת גם מסיביות של ההוראה הממתינה כבר באוגר).
 ממצא זה מדייק את ההשערה המבנית שהוצגה לעיל: המסלול הקובע יושב בפועל בשלב ה-EX (לא ב-ID כפי שהוצע
 כמועמד), ומעורבת בו ישירות לוגיקת הבחירה בין ה-ALU ל-MULT_1.

השוואה לערך התיאורטי

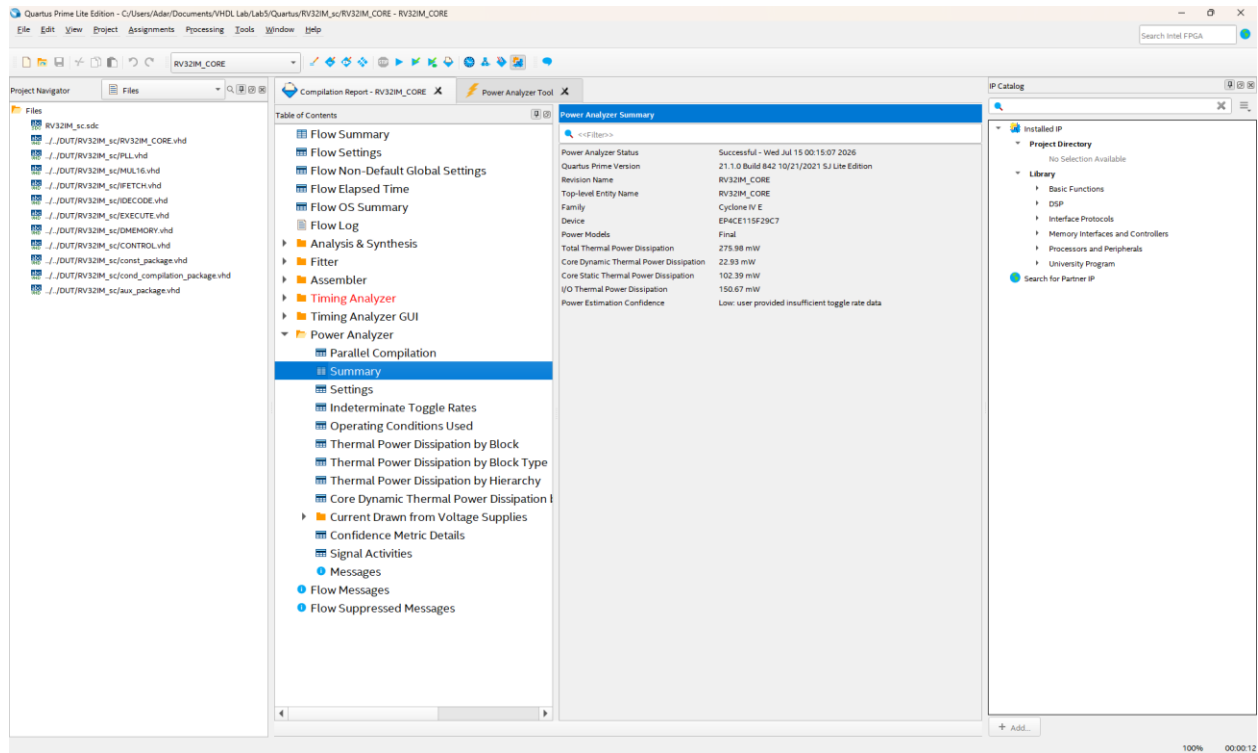
לפי המשימה, תכן pipeline איכותי אמור להתקרב ל-

$$f_{\max}^{\text{pipeline}} = 5 \cdot f_{\max}^{\text{single-cycle}} = 5 \cdot 26.81 = 134.05 \text{ MHz}$$

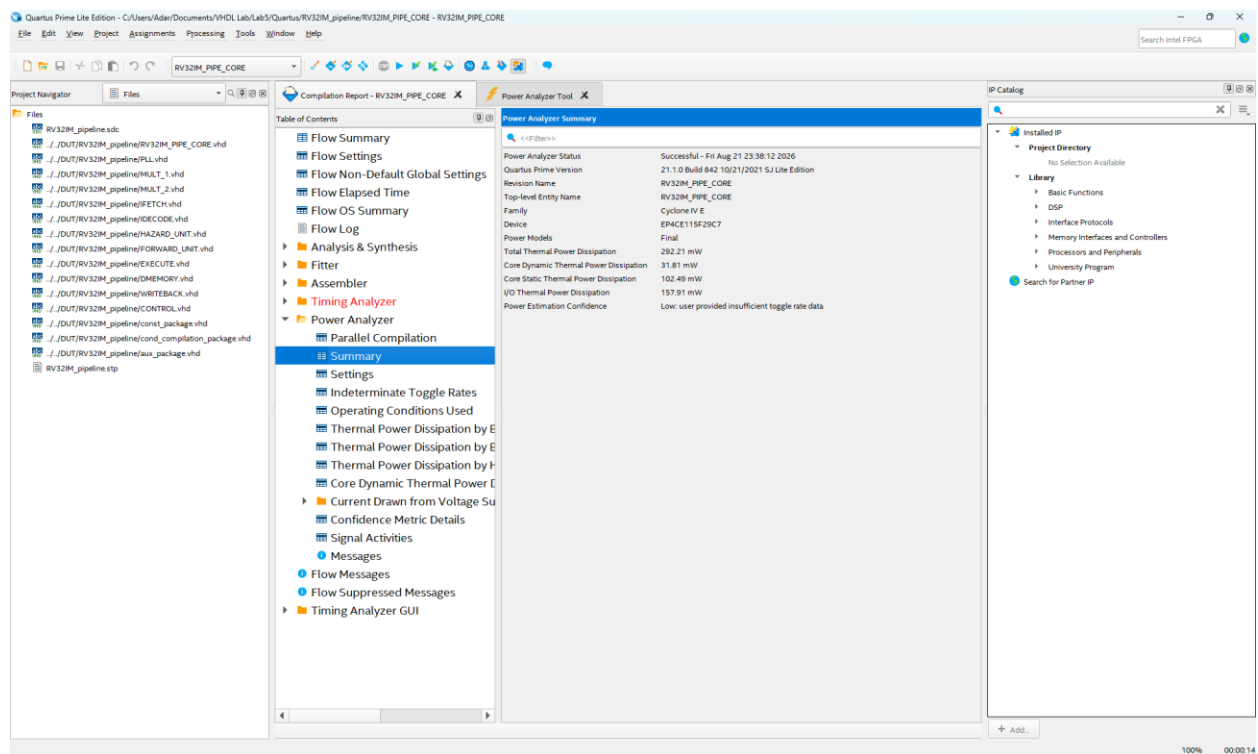
בפועל התקבלו 51.7 MHz, כלומר שיפור של $1.56 \times$ בלבד ביחס ל-single-cycle. הפער נובע מארבע סיבות מבניות:

1. ההנחה שמאחורי הכפולה $5 \times$ היא שניתן לחלק את המסלול לחמישה חלקים שווים. בפועל, השהיית הגישה לבלוקי ה-M9K היא רכיב קשיח שאינו ניתן לחיתוך על ידי הוספת אוגרים, והיא מהווה רצפת השהייה ששילבי ה-IF וה-MEM אינם יכולים לרדת מתחתיה.
2. הכתיבה על השעון ההפוך נשמרה גם ב-pipeline, ולכן המסלול מאוגר ה-EX/MEM אל כתובת ה-DTCM נותר מסלול חצי-מחזור. הוא לבדו כבר מגביל את התכן ל- $\approx 97.8 \text{ MHz}$, הרבה מתחת ל- 134 MHz התיאורטיים.
3. השלבים אינם מאוזנים ביניהם. שלב ה-ID מרכז קריאה מקובץ אוגרים 32×32 , פענוח מלא, ייצור חמש תבניות immediate ובדיקת hazards; זהו כנראה השלב האיטי שקובע את התדר.
4. ל-pipeline עצמו יש תקורה: כל שלב משלם clock-to-Q של אוגר ה-pipeline ו-setup של האוגר הבא, ובנוסף אי-ודאות השעון. תקורה זו אינה מתחלקת ב-5.

Power 6.4: צריכת הספק



איור 48: דוח Power Analyzer Summary של מימוש ה-single-cycle



איור 49: Power Analyzer Summary של מימוש ה-pipeline

7. ניתוח IPC על בסיס אפליקציות benchmark

סעיף 8.c.iii של המשימה דורש לאמת את ה-IPC בהשוואה בין שני אגפי המשוואה שבסעיף 4.5. אגף אחד מחושב ממדני החומרה, והאגף השני הוא מספר ההוראות שהאפליקציה מבצעת בפועל. כדי שההשוואה תהיה משמעותית, מספר ההוראות הזה צריך להיות ידוע במדויק.

7.1 מקור בלתי-תלוי למספר ההוראות

לשם כך נעזרנו במאפיין שתואר בסעיף 3.3. במימוש ה-single-cycle מתקיים $IPC = 1$ בהגדרה, ולכן מונה מחזורי השעון שלו בסוף הריצה שווה בדיוק למספר ההוראות שהאפליקציה מבצעת. שני המימושים מריצים את אותו קוד בינארי ומייצרים DTCM זהה (סעיף 5.2), ולכן מונה המחזוריים שלו משמש לנו מודל זהב לבדיקת מונה ההוראות של ה-pipeline.

IPC	הוראות שפרשו $CLKCNT-(STCNT+4+3 \cdot FHCNT)$	FHCNT	STCNT	CLKCNT	מחזורי single-cycle (= מספר ההוראות)	benchmark
0.807	134	6	10	166	134	test1
0.7905	1,513	100	100	1,914	1,514	test2
0.7519	2,722	298	0	3,620	2,725	test3
0.7489	2,732	304	0	3,648	2,735	test4

טבלה 15: מוני החומרה וחישוב ה-IPC לאפליקציות ה-benchmark

אימות המשוואה

בכל ארבעת הטסטים מספר ההוראות שהתקבל מהמונים קטן ממודל הזהב בדיוק ב-3, למשל ב-test1: $167 - 131 = 36$, $131 = (8 + 4 + 3 \cdot 8)$, לעומת 134 במודל הזהב, ועבור test3: $2722 = (0 + 4 + 3 \cdot 298)$, $3620 - 2725 = 895$. ההפרש הקבוע של $depth = 3$ בדיוק, בכל ארבעת הטסטים ללא יוצא מן הכלל, מצביע על מקור שיטתי ולא על שגיאת מדידה אקראית. ה-flush ה"אחרון" שבצילום המסך הוא כבר האיטרציה הראשונה של לולאת הסיום האינסופית, לא אירוע flush אמיתי בגוף התוכנית, ונוסחת ה-IPC "מקנסת" אותו פי $depth=3$ מבלי שהוא תרם הוראות ממשיות. בהתעלם מהפרש שיטתי וקבוע זה, ההתאמה בין שני אגפי המשוואה מצוינת ומאמתת הן את המונים והן את הקבועים 4 (מילוי ה-pipeline) ו-3 (עומק ה-flush) שבנוסחה.

7.2 זמן ריצה בפועל

ערך IPC קטן מ-1 אינו מעיד שה-pipeline נחותה; הוא מתאר את יעילות המחזורים בלבד. זמן הריצה בפועל הוא מכפלה של מספר המחזורים בתקופת השעון:

$$T_{\text{exec}} = \text{Cycles} \times T_{\text{clk}} = \text{Cycles} / f$$

שיפור	pipeline: מחזורים $\times 19.34 \text{ ns}$	SC: מחזורים $\times 37.30 \text{ ns}$	benchmark
1.55×	167→3.23us	134→5us	test1
1.52×	1915→37.04us	1514→56.47us	test2
1.45×	3620→70.01us	2725→101.64us	test3
1.45×	3648→70.56us	2735→102.02us	test4

טבלה 16: זמן ריצה בפועל כאשר כל מימוש פועל בתדר המרבי שלו (26.81 ו-51.7 MHz בהתאמה)

השיפור בפועל, פי 1.45 עד פי 1.55, נמוך משיפור התדר לבדו (פי 1.93) משום שה IPC של ה-pipeline קטן מ-1 (כ-0.75–0.79), בעוד ש-Single-Cycle תמיד $\text{IPC}=1$. פורמלית: $\text{speedup} = \text{fmax_ratio} \times \text{IPC_pipeline}$; עבור test2 למשל $1.52 \approx 0.789 \times 1.93$, בהתאמה טובה לערך שחושב ישירות מהמחזורים.

בתצורה הנוכחית, שני המימושים מוגדרים לאותו יחס PLL ($G_PLL_DIV=2$, $G_PLL_MUL=1$), ולכן בפועל שניהם עובדים ב-25 MHz בלבד. בתדר זה זמן הריצה נקבע לפי מספר המחזורים בלבד, ולכן ה-pipeline יוצא דווקא איטי יותר (למשל ב-test2: $1,915/25 \text{ MHz} = 76.60 \mu\text{s}$ מול $1,514/25 \text{ MHz} = 60.56 \mu\text{s}$ ב-Single-Cycle). כדי לממש את הפוטנציאל שחושב לעיל יש להעלות את יחס ה-PLL של ה-pipeline. מאחר שנמדד fmax של 51.7 MHz, קיים headroom משמעותי מעל תדר העבודה הנוכחי (25 MHz): ניתן להשתמש למשל ביחס הפשוט $G_PLL_MUL=1$, $G_PLL_DIV=1$ (50 MHz) – כמעט כפול מתדר העבודה הנוכחי, ועדיין מתחת ל- fmax הנמדד.

8. סיכום ומסקנות

בעבודה זו תכננו שני מימושים של מעבד RV32IM תואם RISC-V, סינתזנו אותם ואימתנו את פעולתם. המימוש הראשון הוא single-cycle, והשני pipeline סקלרי בן חמישה שלבים עם forwarding מלא, load--interlock ל-load, use, הכרעת הסתעפויות ב-MEM, וכופל בן 16 ביט המפוצל לשני שלבים.

תוצאות עיקריות

- ארבע אפליקציות ה-benchmark הופקו בשני המימושים, ותמונות ה-DTCM שהתקבלו זהות למודל הזהב של RARS, וגם זהות בית-בית זו לזו. שני המעבדים נכונים אפוא מבחינה פונקציונלית, כולל פיצול הכפל לשני שלבים.
- הכופל מופה כמצופה: ארבעה כופלים משובצים בני 9 ביט נוצלו בפועל בשני המימושים (ב-Single-Cycle בתוך MUL16, וב-pipeline בתוך MULT_1), בהתאם לאלגוריתם הנדרש.
- נוסחת ה-IPC אומתה על ארבעת הטסטים: מונה ההוראות המחושב מהמונים קטן ממספר ההוראות שנמדד ב-Single-Cycle בהפרש קבוע של 3 (depth=) בכל ארבעת הטסטים – פער שיטתי הנובע מרגע העצירה בקצה לולאת הסיום (סעיף 7.2), ולא משגיאה אקראית.

מסקנות PPA

- ה-pipeline קונה ביצועים באוגרים: +46.8% אוגרים לעומת +13.4% לוגיקה בלבד. כך נראית המרה של השהיה לתפוקה.
- מחיר ההספק (+5.9% סך הכול, +38.7% בהספק הדינמי) קטן משמעותית ביחס לשיפור התדר (פי 1.93), ולכן יחס הביצועים-להספק השתפר.
- פיצול הכפל לשני שלבים (MULT_1/MULT_2) לא רק תיקן את הארכיטקטורה להתאמה לאיור 7 של המשימה, אלא גם הקל את שלב ה-EX והעלה את fmax (מ-מסלול מלא הכולל MUL16 ב-EX, אל שלב EX קל יותר עם ארבעה מכפלים חלקיים בלבד).
- הגישה ל-135 MHz (fmax_SC×5) עדיין רחוקה (הושג כ-38.6% מהתיאורטי): רצפת ההשהיה של הזיכרון המשובץ, אי-איזון בין השלבים (ID עדיין הכבד ביותר), והתקורה המובנית של אוגרי ה-pipeline ממשיכים להגביל.